# EN4554 - Deep Learning for Vision

## Assignment 2: Convolutional Neural Networks

**Name:** Savindu Dilshan Wickramasinghe

**Index number:** 220701X

**GitHub repository:** https://github.com/Savidilsh/EN4554-Assignment2-CNNs

**AI assistance acknowledgment:** I used Claude (Anthropic) to help write and debug the code in this notebook. I reviewed and understood all of the code and the results.

## Setup

The notebook runs on a Colab T4 or a local GPU. I produced the outputs below on an NVIDIA GTX 1650. I keep the normalized dataset on the GPU and do batching and augmentation with tensor operations. This is faster than a CPU `DataLoader`. Mixed precision runs only on GPUs with tensor cores. Each finished training run is saved in `results/`. When I run the notebook again, it loads these runs instead of training them again.

In [1]:
```
import copy, json, math, os, random, time, warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from IPython.display import display

warnings.filterwarnings("ignore", message=".*lr_scheduler.step.*")
pd.set_option("display.precision", 4)

SEED = 0

try:  # on Colab, keep data and results on Google Drive
    from google.colab import drive
    drive.mount("/content/drive")
    BASE = "/content/drive/MyDrive/EN4554_A2"
except ImportError:
    BASE = "."
DATA_DIR = os.path.join(BASE, "data")
RESULTS = os.path.join(BASE, "results")
FIGS = os.path.join(BASE, "figures")
```

```
CKPT = os.path.join(BASE, "checkpoints")
for d in (DATA_DIR, RESULTS, FIGS, CKPT):
    os.makedirs(d, exist_ok=True)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# fp16 autocast only on GPUs with tensor cores
AMP = (device.type == "cuda" and torch.cuda.get_device_capability()[0] >= 7
       and "GTX" not in torch.cuda.get_device_name(0))
torch.backends.cudnn.benchmark = True


def set_seed(seed=SEED):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)


def savefig(name):
    plt.savefig(f"{FIGS}/{name}.png", dpi=130, bbox_inches="tight")


_LOG = None


def log(msg):
    """Print a training log line and keep it with the saved run."""
    print(msg, flush=True)
    if _LOG is not None:
        _LOG.append(msg)


def cached(name, fn):
    """Run fn() once and save the result with its training log in
RESULTS/name.json.
    If the run already exists, load it and print its saved training log."""
    global _LOG
    path = f"{RESULTS}/{name}.json"
    if os.path.exists(path):
        with open(path) as f:
            out = json.load(f)
        print(f"Loaded saved run: {name}")
        for line in out.get("_log", []):
            print(line)
        return out
    _LOG = []
    try:
        out = fn()
        out["_log"] = _LOG
    finally:
        _LOG = None
    with open(path, "w") as f:
        json.dump(out, f)
    return out


set_seed()
print("Device:", device, torch.cuda.get_device_name(0) if device.type == "cuda"
```

```
else "", "| mixed precision:", AMP)
print("PyTorch", torch.__version__)
```

```
Device: cuda NVIDIA GeForce GTX 1650 | mixed precision: False
PyTorch 2.11.0+cu128
```

## Data

I split the CIFAR-10 training set into 45,000 training images and 5,000 validation images with seed 0. I compute the mean and standard deviation of each channel from the 45,000 training images only. The test set stays on the CPU. I use it only once, for the final evaluation in Task 4.

In [2]:
```
train_set = torchvision.datasets.CIFAR10(DATA_DIR, train=True, download=True)
test_set = torchvision.datasets.CIFAR10(DATA_DIR, train=False, download=True)
CLASSES = train_set.classes

X_all = torch.from_numpy(train_set.data).permute(0, 3, 1, 2).contiguous()  #
uint8, N x 3 x 32 x 32
y_all = torch.tensor(train_set.targets)
perm = torch.randperm(len(y_all), generator=torch.Generator().manual_seed(SEED))
tr_idx, va_idx = perm[:45000], perm[45000:]

# per-channel statistics of the 45k training images only
MEAN = torch.stack([X_all[tr_idx, c].float().div(255).mean() for c in range(3)])
STD = torch.stack([X_all[tr_idx, c].float().div(255).std() for c in range(3)])


def normalize(X):
    return (X.float().div(255) - MEAN[:, None, None]) / STD[:, None, None]


X_tr, y_tr = normalize(X_all[tr_idx]).to(device), y_all[tr_idx].to(device)
X_va, y_va = normalize(X_all[va_idx]).to(device), y_all[va_idx].to(device)
# the test set stays on the CPU and is only used once, in Task 4
X_te = normalize(torch.from_numpy(test_set.data).permute(0, 3, 1, 2))
y_te = torch.tensor(test_set.targets)

print("train", tuple(X_tr.shape), "val", tuple(X_va.shape), "test",
tuple(X_te.shape))
print("per-channel mean (R, G, B):", MEAN.numpy().round(4), " std:",
STD.numpy().round(4))
display(pd.DataFrame({"train (45k)": torch.bincount(y_tr.cpu()).tolist(),
                      "validation (5k)": torch.bincount(y_va.cpu()).tolist()},
index=CLASSES).T)

# a few (un-normalized) training images
fig, axes = plt.subplots(2, 10, figsize=(15, 3.4))
for k, ax in enumerate(axes.flat):
    i = int(tr_idx[k])
    ax.imshow(train_set.data[i]); ax.set_title(CLASSES[train_set.targets[i]],
fontsize=8); ax.axis("off")
plt.suptitle("Sample CIFAR-10 training images"); plt.tight_layout(); plt.show()
```

```
train (45000, 3, 32, 32) val (5000, 3, 32, 32) test (10000, 3, 32, 32)
per-channel mean (R, G, B): [0.4912 0.482  0.4464]  std: [0.2472 0.2436 0.2617]
```

| | airplane | automobile | bird | cat | deer | dog | frog | horse | ship | truck |
|---|---|---|---|---|---|---|---|---|---|---|
| **train (45k)** | 4518 | 4492 | 4506 | 4496 | 4468 | 4493 | 4498 | 4494 | 4502 | 4533 |
| **validation (5k)** | 482 | 508 | 494 | 504 | 532 | 507 | 502 | 506 | 498 | 467 |




## Training utilities

`flip_crop` , `cutout` and `mixup` work on batches on the GPU. `fit` trains a model. After each epoch it records the training loss, the validation loss and the validation accuracy. It can also record the accuracy on the clean training images.

In [3]:
```
def batches(X, y, bs, shuffle=True):
    n = len(y)
    idx = torch.randperm(n, device=X.device) if shuffle else torch.arange(n,
device=X.device)
    for i in range(0, n, bs):
        j = idx[i:i + bs]
        yield X[j], y[j]


def flip_crop(x, pad=4):
    """Random horizontal flip + random 32x32 crop from a zero-padded (= mean-
padded) image."""
    N, C, H, W = x.shape
    flip = torch.rand(N, device=x.device) < 0.5
    x = torch.where(flip[:, None, None, None], x.flip(3), x)
    xp = F.pad(x, (pad, pad, pad, pad))
    i = torch.randint(0, 2 * pad + 1, (N,), device=x.device)
    j = torch.randint(0, 2 * pad + 1, (N,), device=x.device)
    rows = (i[:, None] + torch.arange(H, device=x.device))[:, None, :, None]
    cols = (j[:, None] + torch.arange(W, device=x.device))[:, None, None, :]
    n_idx = torch.arange(N, device=x.device)[:, None, None, None]
    c_idx = torch.arange(C, device=x.device)[None, :, None, None]
    return xp[n_idx, c_idx, rows, cols]


def cutout(x, size=16):
    """Zero (= mean) a size x size square at a random centre of each image (may be
clipped at the border)."""
    N, _, H, W = x.shape
    cy = torch.randint(0, H, (N, 1, 1), device=x.device)
    cx = torch.randint(0, W, (N, 1, 1), device=x.device)
    yy = torch.arange(H, device=x.device)[None, :, None]
```

```
    xx = torch.arange(W, device=x.device)[None, None, :]
    mask = (yy >= cy - size // 2) & (yy < cy + size // 2) & (xx >= cx - size // 2)
& (xx < cx + size // 2)
    return x * (~mask)[:, None].to(x.dtype)


def mixup(x, y, alpha):
    """Return mixed inputs, both label sets and the mixing coefficient lambda ~
Beta(alpha, alpha)."""
    lam = float(np.random.beta(alpha, alpha))
    p = torch.randperm(len(y), device=x.device)
    return lam * x + (1 - lam) * x[p], y, y[p], lam


def compose(*fs):
    def f(x):
        for g in fs:
            x = g(x)
        return x
    return f


def cosine_warmup(opt, total_steps, warmup_steps=0):
    """Per-iteration LR multiplier: linear warm-up, then cosine decay to 0."""
    def mult(s):
        if s < warmup_steps:
            return (s + 1) / warmup_steps
        p = (s - warmup_steps) / max(1, total_steps - warmup_steps)
        return 0.5 * (1 + math.cos(math.pi * min(p, 1.0)))
    return torch.optim.lr_scheduler.LambdaLR(opt, mult)


@torch.no_grad()
def evaluate(model, X, y, bs=1000):
    model.eval()
    n = len(y)
    loss, correct = 0.0, 0
    for i in range(0, n, bs):
        xb, yb = X[i:i + bs].to(device), y[i:i + bs].to(device)
        with torch.autocast(device.type, dtype=torch.float16, enabled=AMP):
            out = model(xb)
        loss += F.cross_entropy(out.float(), yb, reduction="sum").item()
        correct += (out.argmax(1) == yb).sum().item()
    return loss / n, correct / n


def fit(model, opt, n_epochs, X, y, bs=128, aug=None, mixup_alpha=0.0, sched=None,
        label_smoothing=0.0, amp=AMP, eval_train=False, tag=""):
    """Train for n_epochs; returns per-epoch train loss / val loss / val acc (and
clean train acc)."""
    scaler = torch.amp.GradScaler(device.type, enabled=amp)
    H = dict(train_loss=[], val_loss=[], val_acc=[], train_acc=[], iter_loss=[],
time=[])
    for ep in range(n_epochs):
        model.train()
        t0, losses = time.time(), []
```

```
        for xb, yb in batches(X, y, bs):
            if aug is not None:
                xb = aug(xb)
            with torch.autocast(device.type, dtype=torch.float16, enabled=amp):
                if mixup_alpha > 0:
                    xb, ya, yb2, lam = mixup(xb, yb, mixup_alpha)
                    out = model(xb)
                    loss = lam * F.cross_entropy(out, ya) + (1 - lam) *
F.cross_entropy(out, yb2)
                else:
                    out = model(xb)
                    loss = F.cross_entropy(out, yb,
label_smoothing=label_smoothing)
            opt.zero_grad(set_to_none=True)
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
            if sched is not None:
                sched.step()
            losses.append(loss.detach().float())
        il = torch.stack(losses).cpu().tolist()
        H["iter_loss"] += il
        H["train_loss"].append(float(np.mean(il)))
        vl, va = evaluate(model, X_va, y_va)
        H["val_loss"].append(vl); H["val_acc"].append(va)
        if eval_train:
            H["train_acc"].append(evaluate(model, X, y)[1])
        H["time"].append(time.time() - t0)
        msg = f"[{tag}] ep {ep + 1:3d}/{n_epochs} loss {H['train_loss'][-1]:.4f}
val_acc {va:.4f}"
        if eval_train:
            msg += f" train_acc {H['train_acc'][-1]:.4f}"
        if ep == 0 or (ep + 1) % max(1, n_epochs // 10) == 0 or ep == n_epochs -
1:
            log(msg + f" ({H['time'][-1]:.1f}s)")
        if not math.isfinite(H["train_loss"][-1]):
            log(f"[{tag}] diverged (non-finite loss), stopping early")
            break
    return H


def n_params(model):
    return sum(p.numel() for p in model.parameters())


def smooth(v, k=50):
    v = np.asarray(v, dtype=float)
    if len(v) < k:
        return v
    return np.convolve(v, np.ones(k) / k, mode="valid")
```

# Task 1: Backpropagation

## 1(a): Manual layers and MLP

Each layer has a `forward` and a `backward` method. `forward` stores the values that `backward` needs. `backward` applies the chain rule. The code does not use autograd.

- **Linear** $Y = XW + b$: $\partial L/\partial W = X^\top G$, $\partial L/\partial b = \sum_n G_n$, $\partial L/\partial X = GW^\top$, where $G = \partial L/\partial Y$.
- **ReLU** $y = \max(0, x)$: $\partial L/\partial x = G \odot 1[x > 0]$.
- **Sigmoid** (used in 1(d)) $s = \sigma(x)$: $\partial L/\partial x = G \odot s(1 - s)$.
- **Softmax cross-entropy**, averaged over the batch: $L = -\frac{1}{N}\sum_n \log p_{n,y_n}$, $\partial L/\partial z = (p - \text{onehot}(y))/N$. The forward pass subtracts the row maximum before the exponential. This keeps the numbers stable.

In [4]:
```
class Linear:
    def __init__(self, d_in, d_out, init="he", dtype=torch.float32, dev=device,
gen=None):
        std = math.sqrt(2.0 / d_in) if init == "he" else math.sqrt(2.0 / (d_in +
d_out))  # He or Xavier
        self.W = torch.randn(d_in, d_out, generator=gen, dtype=dtype).to(dev) *
std
        self.b = torch.zeros(d_out, dtype=dtype, device=dev)
        self.dW = self.db = None

    def forward(self, x):
        self.x = x
        return x @ self.W + self.b

    def backward(self, g):
        self.dW = self.x.T @ g
        self.db = g.sum(0)
        return g @ self.W.T

    def params(self):
        return [(self.W, self.dW), (self.b, self.db)]


class ReLU:
    def forward(self, x):
        self.mask = x > 0
        return x * self.mask

    def backward(self, g):
        return g * self.mask

    def params(self):
        return []


class Sigmoid:
    def forward(self, x):
        self.s = 1.0 / (1.0 + torch.exp(-x))
        return self.s

    def backward(self, g):
        return g * self.s * (1 - self.s)
```

```
    def params(self):
        return []


class SoftmaxCrossEntropy:
    def forward(self, z, y):
        z = z - z.max(dim=1, keepdim=True).values
        logp = z - torch.log(torch.exp(z).sum(dim=1, keepdim=True))
        self.p, self.y = torch.exp(logp), y
        return -logp[torch.arange(len(y)), y].mean()

    def backward(self):
        g = self.p.clone()
        g[torch.arange(len(self.y)), self.y] -= 1
        return g / len(self.y)


class MLP:
    def __init__(self, sizes, act="relu", init="he", dtype=torch.float32,
dev=device, seed=SEED):
        gen = torch.Generator().manual_seed(seed)
        Act = ReLU if act == "relu" else Sigmoid
        self.layers = []
        for i, (a, b) in enumerate(zip(sizes[:-1], sizes[1:])):
            self.layers.append(Linear(a, b, init, dtype, dev, gen))
            if i < len(sizes) - 2:
                self.layers.append(Act())
        self.loss_fn = SoftmaxCrossEntropy()

    def forward(self, x):
        for layer in self.layers:
            x = layer.forward(x)
        return x

    def loss(self, x, y):
        return self.loss_fn.forward(self.forward(x), y)

    def backward(self):
        g = self.loss_fn.backward()
        for layer in reversed(self.layers):
            g = layer.backward(g)
        return g  # dL/dx

    def linears(self):
        return [l for l in self.layers if isinstance(l, Linear)]

    def sgd_step(self, lr):
        for lin in self.linears():
            lin.W -= lr * lin.dW
            lin.b -= lr * lin.db
```

## 1(b): Gradient check

I use a small MLP (12-10-8-5) in float64 and a random batch of 6 inputs. I compare every parameter gradient and the input gradient with two references: autograd, and centred finite differences $\big(L(\theta + he_i) - L(\theta - he_i)\big)/2h$ with $h = 10^{-6}$. The relative error is $\|g_m - g\|_2 / (\|g_m\|_2 + \|g\|_2)$. I run the check for a ReLU network and a sigmoid network.

In [5]:
```
def rel_err(a, b):
    return ((a - b).norm() / max((a.norm() + b.norm()).item(), 1e-12)).item()


def grad_check(act):
    torch.manual_seed(1)
    net = MLP([12, 10, 8, 5], act=act, dtype=torch.float64, dev="cpu", seed=1)
    for lin in net.linears():  # non-zero biases so that their gradients are
tested properly
        lin.b += 0.1 * torch.randn_like(lin.b)
    x = torch.randn(6, 12, dtype=torch.float64)
    y = torch.randint(0, 5, (6,))

    # manual gradients
    net.loss(x, y)
    dx_manual = net.backward()
    manual = {}
    for k, lin in enumerate(net.linears()):
        manual[f"W{k + 1}"], manual[f"b{k + 1}"] = lin.dW.clone(), lin.db.clone()
    manual["x"] = dx_manual

    # autograd on the same weights
    leaves = {}
    h = x.clone().requires_grad_(True)
    leaves["x"] = h
    lins = net.linears()
    for k, lin in enumerate(lins):
        W = lin.W.clone().requires_grad_(True); b =
lin.b.clone().requires_grad_(True)
        leaves[f"W{k + 1}"], leaves[f"b{k + 1}"] = W, b
        h = h @ W + b
        if k < len(lins) - 1:
            h = torch.relu(h) if act == "relu" else torch.sigmoid(h)
    F.cross_entropy(h, y).backward()

    # centred finite differences
    def fd(tensor, eps=1e-6):
        g = torch.zeros_like(tensor)
        flat, gflat = tensor.view(-1), g.view(-1)
        for i in range(flat.numel()):
            old = flat[i].item()
            flat[i] = old + eps; lp = net.loss(x, y).item()
            flat[i] = old - eps; lm = net.loss(x, y).item()
            flat[i] = old
            gflat[i] = (lp - lm) / (2 * eps)
        return g

    rows = []
    for name, g_m in manual.items():
```

```
        if name == "x":
            target = x
        else:
            k = int(name[1:]) - 1
            target = lins[k].W if name[0] == "W" else lins[k].b
        g_auto = leaves[name].grad
        g_fd = fd(target)
        rows.append((act, name, tuple(g_m.shape), rel_err(g_m, g_auto),
rel_err(g_m, g_fd),
                     torch.allclose(g_m, g_auto), torch.allclose(g_m, g_fd,
atol=1e-7)))
    return rows


gc = pd.DataFrame([r for act in ("relu", "sigmoid") for r in grad_check(act)],
                  columns=["activation", "gradient", "shape", "rel. error vs
autograd", "rel. error vs FD",
                           "allclose (autograd)", "allclose (FD)"])
display(gc.style.format({"rel. error vs autograd": "{:.2e}", "rel. error vs FD": "
{:.2e}"}).hide(axis="index"))
print("all gradients match:", bool(gc["allclose (autograd)"].all() and
gc["allclose (FD)"].all()))
```

| activation | gradient | shape | rel. error vs autograd | rel. error vs FD | allclose (autograd) | allclose (FD) |
|---|---|---|---|---|---|---|
| relu | W1 | (12, 10) | 7.31e-17 | 6.95e-10 | True | True |
| relu | b1 | (10,) | 5.76e-17 | 5.73e-10 | True | True |
| relu | W2 | (10, 8) | 7.16e-17 | 4.20e-10 | True | True |
| relu | b2 | (8,) | 1.03e-16 | 6.51e-10 | True | True |
| relu | W3 | (8, 5) | 3.98e-17 | 3.09e-10 | True | True |
| relu | b3 | (5,) | 1.61e-17 | 3.77e-10 | True | True |
| relu | x | (6, 12) | 5.83e-17 | 6.39e-10 | True | True |
| sigmoid | W1 | (12, 10) | 1.35e-16 | 7.65e-09 | True | True |
| sigmoid | b1 | (10,) | 1.62e-16 | 1.24e-08 | True | True |
| sigmoid | W2 | (10, 8) | 1.09e-16 | 3.10e-09 | True | True |
| sigmoid | b2 | (8,) | 9.99e-17 | 1.13e-09 | True | True |
| sigmoid | W3 | (8, 5) | 3.04e-17 | 5.72e-10 | True | True |
| sigmoid | b3 | (5,) | 3.54e-17 | 4.01e-10 | True | True |
| sigmoid | x | (6, 12) | 1.28e-16 | 7.63e-09 | True | True |

```
all gradients match: True
```

**Interpretation:** All manual gradients match autograd. The relative errors are about **1e-16**. This is just rounding in float64. The errors against finite differences are **1e-10 to 1e-8**. Finite differences are only approximate, so this is the expected size: the truncation error is of order $h^2$ and the rounding error is of order $\epsilon/h$. `torch.allclose` returns True for every parameter and for the input gradient, for both activations. So the backward passes are correct.

## 1(c): Training the manual MLP

The MLP is 3072-512-256-10 with ReLU and He initialization. I train it with mini-batch SGD: batch size 128, learning rate 0.05 with cosine decay, 30 epochs. The training uses only the manual forward and backward passes from 1(a).

In [6]:
```
def train_manual_mlp(n_epochs=30, lr0=0.05, bs=128):
    set_seed()
    net = MLP([3072, 512, 256, 10], act="relu", init="he")
    Xf_tr, Xf_va = X_tr.reshape(len(X_tr), -1), X_va.reshape(len(X_va), -1)
    steps = n_epochs * math.ceil(len(y_tr) / bs)
    H = dict(iter_loss=[], train_loss=[], val_loss=[], val_acc=[], train_acc=[])
    step = 0
    for ep in range(n_epochs):
        losses = []
        for xb, yb in batches(Xf_tr, y_tr, bs):
            lr = lr0 * 0.5 * (1 + math.cos(math.pi * step / steps))
            losses.append(net.loss(xb, yb))
            net.backward()
            net.sgd_step(lr)
            step += 1
        il = torch.stack(losses).cpu().tolist()
        H["iter_loss"] += il
        H["train_loss"].append(float(np.mean(il)))

        def acc_loss(Xf, y):
            out = torch.cat([net.forward(Xf[i:i + 1000]) for i in range(0, len(y),
1000)])
            return net.loss_fn.forward(out, y).item(), (out.argmax(1) ==
y).float().mean().item()

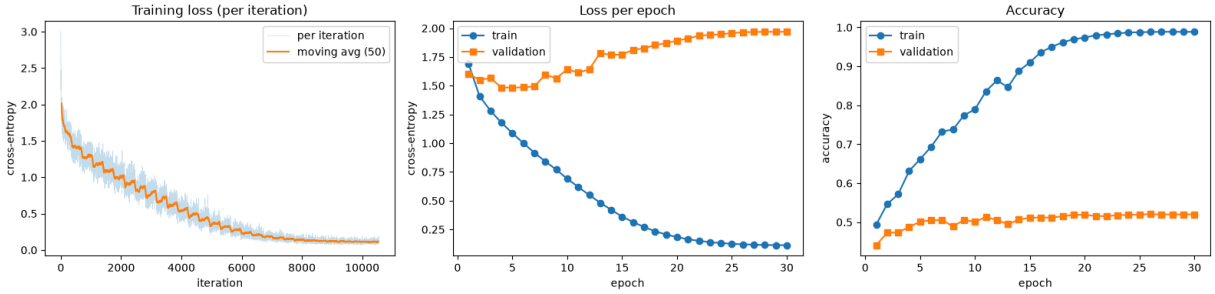
        vl, va = acc_loss(Xf_va, y_va)
        H["val_loss"].append(vl); H["val_acc"].append(va)
        H["train_acc"].append(acc_loss(Xf_tr, y_tr)[1])
        log(f"ep {ep + 1:2d} train_loss {H['train_loss'][-1]:.4f} train_acc
{H['train_acc'][-1]:.4f} "
            f"val_loss {vl:.4f} val_acc {va:.4f}")
    return H


H1c = cached("task1c_mlp", lambda: train_manual_mlp(30))

fig, ax = plt.subplots(1, 3, figsize=(16, 4))
ax[0].plot(H1c["iter_loss"], alpha=0.25, lw=0.5, label="per iteration")
ax[0].plot(np.arange(len(smooth(H1c["iter_loss"]))) + 25,
smooth(H1c["iter_loss"]), label="moving avg (50)")
ax[0].set(xlabel="iteration", ylabel="cross-entropy", title="Training loss (per
iteration)"); ax[0].legend()
ep = np.arange(1, len(H1c["train_loss"]) + 1)
ax[1].plot(ep, H1c["train_loss"], "o-", label="train"); ax[1].plot(ep,
H1c["val_loss"], "s-", label="validation")
ax[1].set(xlabel="epoch", ylabel="cross-entropy", title="Loss per epoch");
ax[1].legend()
ax[2].plot(ep, H1c["train_acc"], "o-", label="train"); ax[2].plot(ep,
```

```
H1c["val_acc"], "s-", label="validation")
ax[2].set(xlabel="epoch", ylabel="accuracy", title="Accuracy"); ax[2].legend()
plt.tight_layout(); savefig("task1c_mlp"); plt.show()
print(f"Final validation accuracy: {H1c['val_acc'][-1]:.4f} (best
{max(H1c['val_acc']):.4f})")
```

```
Loaded saved run: task1c_mlp
ep  1 train_loss 1.6921 train_acc 0.4939 val_loss 1.6016 val_acc 0.4410
ep  2 train_loss 1.4097 train_acc 0.5465 val_loss 1.5511 val_acc 0.4730
ep  3 train_loss 1.2818 train_acc 0.5716 val_loss 1.5657 val_acc 0.4734
ep  4 train_loss 1.1786 train_acc 0.6315 val_loss 1.4843 val_acc 0.4878
ep  5 train_loss 1.0869 train_acc 0.6616 val_loss 1.4841 val_acc 0.5012
ep  6 train_loss 1.0011 train_acc 0.6935 val_loss 1.4861 val_acc 0.5048
ep  7 train_loss 0.9169 train_acc 0.7329 val_loss 1.4939 val_acc 0.5058
ep  8 train_loss 0.8416 train_acc 0.7376 val_loss 1.5961 val_acc 0.4890
ep  9 train_loss 0.7701 train_acc 0.7745 val_loss 1.5643 val_acc 0.5058
ep 10 train_loss 0.6905 train_acc 0.7895 val_loss 1.6438 val_acc 0.5006
ep 11 train_loss 0.6193 train_acc 0.8361 val_loss 1.6131 val_acc 0.5136
ep 12 train_loss 0.5499 train_acc 0.8638 val_loss 1.6423 val_acc 0.5052
ep 13 train_loss 0.4822 train_acc 0.8467 val_loss 1.7841 val_acc 0.4948
ep 14 train_loss 0.4214 train_acc 0.8883 val_loss 1.7671 val_acc 0.5064
ep 15 train_loss 0.3618 train_acc 0.9102 val_loss 1.7720 val_acc 0.5118
ep 16 train_loss 0.3144 train_acc 0.9363 val_loss 1.8113 val_acc 0.5118
ep 17 train_loss 0.2720 train_acc 0.9500 val_loss 1.8249 val_acc 0.5120
ep 18 train_loss 0.2347 train_acc 0.9618 val_loss 1.8541 val_acc 0.5150
ep 19 train_loss 0.2065 train_acc 0.9696 val_loss 1.8695 val_acc 0.5190
ep 20 train_loss 0.1836 train_acc 0.9738 val_loss 1.8924 val_acc 0.5188
ep 21 train_loss 0.1651 train_acc 0.9798 val_loss 1.9101 val_acc 0.5160
ep 22 train_loss 0.1510 train_acc 0.9820 val_loss 1.9350 val_acc 0.5154
ep 23 train_loss 0.1400 train_acc 0.9847 val_loss 1.9417 val_acc 0.5182
ep 24 train_loss 0.1316 train_acc 0.9867 val_loss 1.9480 val_acc 0.5184
ep 25 train_loss 0.1257 train_acc 0.9877 val_loss 1.9570 val_acc 0.5198
ep 26 train_loss 0.1210 train_acc 0.9882 val_loss 1.9646 val_acc 0.5204
ep 27 train_loss 0.1178 train_acc 0.9886 val_loss 1.9673 val_acc 0.5198
ep 28 train_loss 0.1155 train_acc 0.9887 val_loss 1.9682 val_acc 0.5198
ep 29 train_loss 0.1142 train_acc 0.9885 val_loss 1.9687 val_acc 0.5200
ep 30 train_loss 0.1136 train_acc 0.9886 val_loss 1.9689 val_acc 0.5200
```




```
Final validation accuracy: 0.5200 (best 0.5204)
```

**Interpretation:** The training loss falls from about 2.3 to **0.11**. The final validation accuracy is **52.0%**. The training accuracy reaches 98.9%, so the model overfits strongly. The validation loss is lowest at epoch 5 (1.48). After that it rises to 1.97, while the validation accuracy stays at 51-52%. So the model becomes more confident on the validation images it gets wrong.

An MLP sees the image as a flat list of pixels. It does not use the local structure of the image, and a small shift changes its input completely. The model has 1.7M parameters, so it

memorises the training images instead of learning general features. The CNNs in the next tasks reach much higher accuracy with fewer parameters.

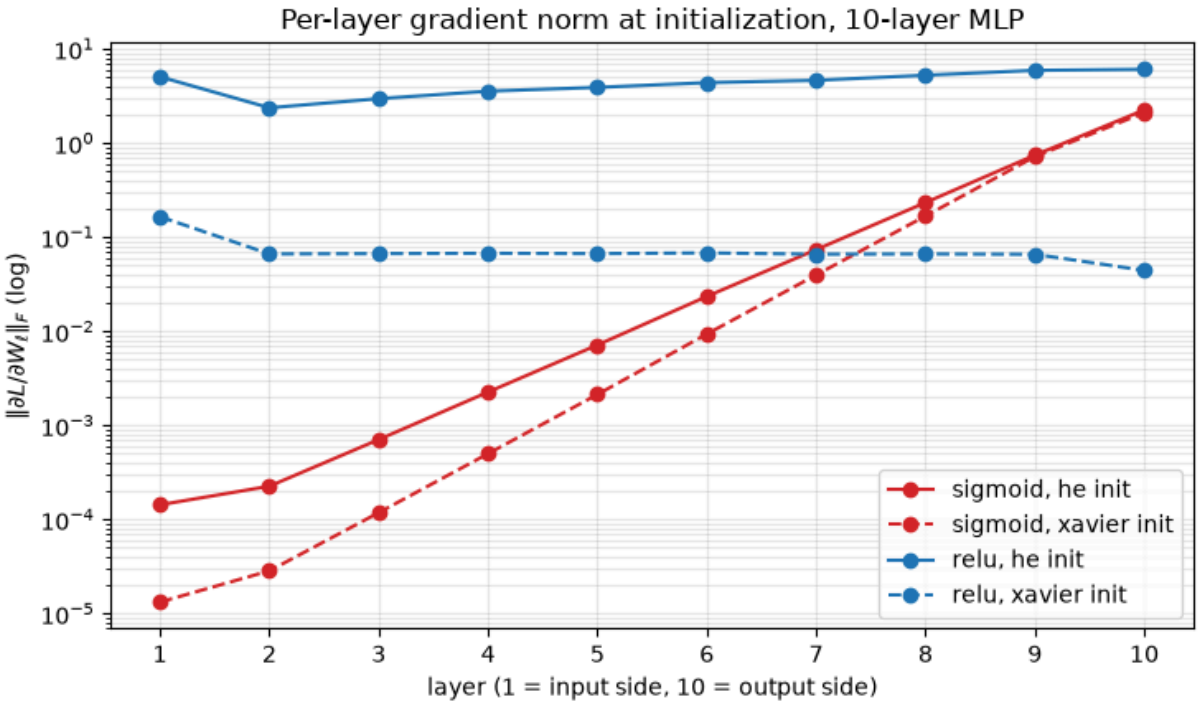
## 1(d): Gradient norms in a 10-layer MLP

The network has 10 linear layers: 3072 inputs, 9 hidden layers of 256 units, and 10 outputs. Layer 1 is next to the input. At initialization, I compute $\|\partial L/\partial W_\ell\|_F$ for each layer on a fixed batch of 512 training images. I do this for sigmoid and for ReLU, with the same weights. I show two initializations: He ($\mathrm{Var}(W) = 2/d_{in}$) and Xavier ($\mathrm{Var}(W) = 2/(d_{in} + d_{out})$). Each curve is the average over 5 seeds.

In [7]:
```
def layer_grad_norms(act, init, seeds=range(5)):
    xb, yb = X_tr[:512].reshape(512, -1), y_tr[:512]
    norms = []
    for s in seeds:
        net = MLP([3072] + [256] * 9 + [10], act=act, init=init, seed=s)
        net.loss(xb, yb)
        net.backward()
        norms.append([lin.dW.norm().item() for lin in net.linears()])
    return np.mean(norms, axis=0)


fig, ax = plt.subplots(figsize=(7.5, 4.5))
gn = {}
for act, c in (("sigmoid", "tab:red"), ("relu", "tab:blue")):
    for init, ls in (("he", "-"), ("xavier", "--")):
        g = gn[f"{act}, {init} init"] = layer_grad_norms(act, init)
        ax.semilogy(range(1, 11), g, ls, color=c, marker="o", label=f"{act},
{init} init")
ax.set(xlabel="layer (1 = input side, 10 = output side)", ylabel=r"$\|\partial
L/\partial W_\ell\|_F$ (log)",
       title="Per-layer gradient norm at initialization, 10-layer MLP")
ax.set_xticks(range(1, 11)); ax.grid(True, which="both", alpha=0.3); ax.legend()
plt.tight_layout(); savefig("task1d_gradnorms"); plt.show()

gn = pd.DataFrame(gn, index=[f"layer {i}" for i in range(1, 11)]).T
gn["layer 1 / layer 10"] = gn["layer 1"] / gn["layer 10"]
display(gn.style.format("{:.1e}"))
```




| | layer 1 | layer 2 | layer 3 | layer 4 | layer 5 | layer 6 | layer 7 | layer 8 | layer 9 | layer 10 | layer 1 / layer 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **sigmoid, he init** | 1.4e-04 | 2.3e-04 | 7.1e-04 | 2.3e-03 | 7.1e-03 | 2.3e-02 | 7.4e-02 | 2.3e-01 | 7.4e-01 | 2.2e+00 | 6.4e-05 |
| **sigmoid, xavier init** | 1.3e-05 | 2.8e-05 | 1.2e-04 | 5.0e-04 | 2.1e-03 | 9.3e-03 | 4.0e-02 | 1.7e-01 | 7.2e-01 | 2.1e+00 | 6.3e-06 |
| **relu, he init** | 5.0e+00 | 2.4e+00 | 3.0e+00 | 3.5e+00 | 3.9e+00 | 4.4e+00 | 4.6e+00 | 5.2e+00 | 5.9e+00 | 6.1e+00 | 8.3e-01 |
| **relu, xavier init** | 1.6e-01 | 6.6e-02 | 6.7e-02 | 6.7e-02 | 6.7e-02 | 6.8e-02 | 6.6e-02 | 6.6e-02 | 6.6e-02 | 4.4e-02 | 3.7e+00 |

**Interpretation:** Backpropagation computes $\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot \phi'(z_\ell)$ and $\partial L/\partial W_\ell = h_{\ell-1}^\top \delta_\ell$.

**Sigmoid:** the derivative is at most 0.25. So the gradient gets smaller at every layer. On the log scale the curve is a straight line. The gradient of layer 1 is about $10^4$ **(He) to** $10^5$ **(Xavier) times smaller** than the gradient of layer 10. The first layers would learn very slowly. This is the vanishing-gradient problem.

**ReLU:** the derivative is 0 or 1, and about half of the units are active. He initialization doubles the weight variance to make up for this. So the gradient norm stays almost constant across the layers. With Xavier initialization the curve is also flat, but about 100 times lower. The activations shrink in the forward pass and $\delta$ shrinks in the backward pass. The two effects cancel in $h^\top \delta$. Layer 1 has a slightly larger norm because $W_1$ has many more entries (3072 × 256).

This is why deep networks usually use ReLU with He initialization.

# Task 2: Optimizers

## 2(a): SGD with momentum and Adam

I implement both as subclasses of `torch.optim.Optimizer`.

- **SGD with momentum** (no dampening, as in PyTorch): $g = \nabla_\theta L + \lambda\theta,\ b_t = \mu b_{t-1} + g$ with $b_1 = g,\ \theta \leftarrow \theta - \eta\, b_t$.
- **Adam**: $m_t = \beta_1 m_{t-1} + (1-\beta_1) g,\ v_t = \beta_2 v_{t-1} + (1-\beta_2) g^2$, bias correction $\hat{m}_t = m_t/(1-\beta_1^t)$, $\hat{v}_t = v_t/(1-\beta_2^t)$, $\theta \leftarrow \theta - \eta\, \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$.

In [8]:
```
class MySGD(torch.optim.Optimizer):
    def __init__(self, params, lr, momentum=0.0, weight_decay=0.0):
        super().__init__(params, dict(lr=lr, momentum=momentum,
weight_decay=weight_decay))

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
        for group in self.param_groups:
            lr, mu, wd = group["lr"], group["momentum"], group["weight_decay"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                g = p.grad if wd == 0 else p.grad + wd * p
                if mu != 0:
                    state = self.state[p]
                    if "momentum_buffer" not in state:
                        state["momentum_buffer"] = g.clone()
                    else:
                        state["momentum_buffer"].mul_(mu).add_(g)
                    g = state["momentum_buffer"]
                p.add_(g, alpha=-lr)
        return loss


class MyAdam(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999), eps=1e-8,
weight_decay=0.0):
        super().__init__(params, dict(lr=lr, betas=betas, eps=eps,
weight_decay=weight_decay))

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
```

```
                loss = closure()
        for group in self.param_groups:
            lr, (b1, b2), eps, wd = group["lr"], group["betas"], group["eps"],
group["weight_decay"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                g = p.grad if wd == 0 else p.grad + wd * p
                state = self.state[p]
                if not state:
                    state["t"] = 0
                    state["m"] = torch.zeros_like(p)
                    state["v"] = torch.zeros_like(p)
                state["t"] += 1
                t, m, v = state["t"], state["m"], state["v"]
                m.mul_(b1).add_(g, alpha=1 - b1)
                v.mul_(b2).addcmul_(g, g, value=1 - b2)
                m_hat = m / (1 - b1 ** t)
                v_hat = v / (1 - b2 ** t)
                p.addcdiv_(m_hat, v_hat.sqrt().add_(eps), value=-lr)
        return loss
```

I train two copies of the same small CNN (float64) for 50 steps on the same CIFAR-10 batches. One copy uses my optimizer and the other uses the PyTorch optimizer. Then I compare the final parameters with `torch.allclose`. The table also shows the largest parameter change. This shows that the parameters really moved.

In [9]:
```
def compare_optimizers(make_mine, make_ref, steps=50):
    torch.manual_seed(0)
    net = nn.Sequential(nn.Conv2d(3, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
                        nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(),
nn.AdaptiveAvgPool2d(1),
                        nn.Flatten(), nn.Linear(16, 10)).double().to(device)
    a, b = copy.deepcopy(net), copy.deepcopy(net)
    oa, ob = make_mine(a.parameters()), make_ref(b.parameters())
    for s in range(steps):
        xb = X_tr[s * 64:(s + 1) * 64].double(); yb = y_tr[s * 64:(s + 1) * 64]
        for model, opt in ((a, oa), (b, ob)):
            opt.zero_grad()
            F.cross_entropy(model(xb), yb).backward()
            opt.step()
    diff = max((pa - pb).abs().max().item() for pa, pb in zip(a.parameters(),
b.parameters()))
    same = all(torch.allclose(pa, pb) for pa, pb in zip(a.parameters(),
b.parameters()))
    moved = max((pa - p0).abs().max().item() for pa, p0 in zip(a.parameters(),
net.parameters()))
    return same, diff, moved


tests = {
    "SGD (no momentum)": (lambda p: MySGD(p, lr=0.05), lambda p:
torch.optim.SGD(p, lr=0.05)),
    "SGD momentum 0.9": (lambda p: MySGD(p, lr=0.05, momentum=0.9),
```

```
                            lambda p: torch.optim.SGD(p, lr=0.05, momentum=0.9)),
    "SGD momentum 0.9 + wd 5e-4": (lambda p: MySGD(p, lr=0.05, momentum=0.9,
weight_decay=5e-4),
                                   lambda p: torch.optim.SGD(p, lr=0.05,
momentum=0.9, weight_decay=5e-4)),
    "Adam": (lambda p: MyAdam(p, lr=1e-3), lambda p: torch.optim.Adam(p, lr=1e-
3)),
    "Adam betas (0.8, 0.99) + wd 1e-3": (lambda p: MyAdam(p, lr=3e-3, betas=(0.8,
0.99), weight_decay=1e-3),
                                         lambda p: torch.optim.Adam(p, lr=3e-3,
betas=(0.8, 0.99),

weight_decay=1e-3)),
}
opt_check = pd.DataFrame([(name, *compare_optimizers(mine, ref)) for name, (mine,
ref) in tests.items()],
                         columns=["optimizer (ours vs torch.optim)", "allclose",
"max |difference|",
                                  "max |parameter change|"])
display(opt_check.style.format({"max |difference|": "{:.2e}", "max |parameter
change|": "{:.2e}"})
        .hide(axis="index"))
```

| optimizer (ours vs torch.optim) | allclose | max \|difference\| | max \|parameter change\| |
|---|---|---|---|
| SGD (no momentum) | True | 0.00e+00 | 6.72e-02 |
| SGD momentum 0.9 | True | 0.00e+00 | 2.74e-01 |
| SGD momentum 0.9 + wd 5e-4 | True | 5.55e-17 | 2.73e-01 |
| Adam | True | 8.33e-17 | 5.32e-02 |
| Adam betas (0.8, 0.99) + wd 1e-3 | True | 8.33e-17 | 1.77e-01 |

**Interpretation:** Both optimizers match PyTorch in all five settings. The largest difference is **0** for SGD and momentum. It is about **1e-16** with weight decay and for Adam. In the same runs the parameters moved by up to 0.27. The small Adam difference comes from rounding: PyTorch applies the bias corrections in a different order, but the result is the same in exact math. The check runs over 50 steps in a row. So it also shows that the stored state (momentum buffer, Adam moments and step count) updates correctly.

## 2(b): Trajectories on $f(x, y) = \frac{1}{2}(x^2 + 50y^2)$

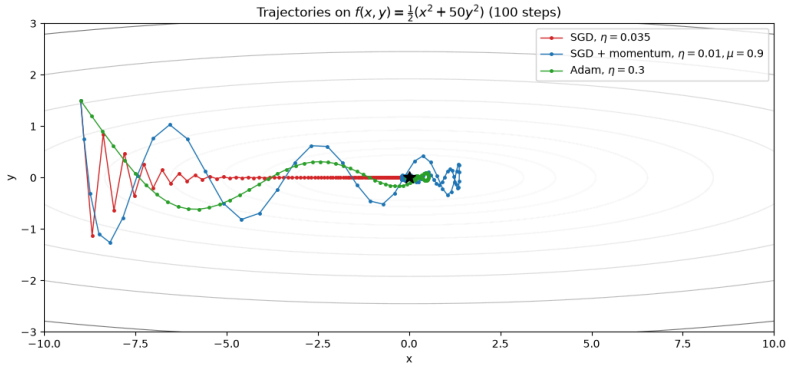
The gradient is $(x,\ 50y)$. So the function is 50 times steeper along $y$ than along $x$. Plain gradient descent is stable only when $\eta < 2/50 = 0.04$. All methods start at $(-9, 1.5)$ and run for 100 updates with my optimizers. SGD uses $\eta = 0.035$, momentum uses $\eta = 0.01$ and $\mu = 0.9$, and Adam uses $\eta = 0.3$.

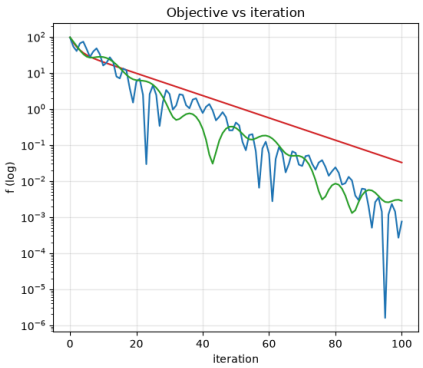
In [10]:
```
def f_quad(p):
    return 0.5 * (p[0] ** 2 + 50 * p[1] ** 2)
```

```
def trajectory(make_opt, start=(-9.0, 1.5), steps=100):
    p = torch.tensor(start, dtype=torch.float64, requires_grad=True)
    opt = make_opt([p])
    path = [p.detach().clone().numpy()]
    for _ in range(steps):
        opt.zero_grad()
        f_quad(p).backward()
        opt.step()
        path.append(p.detach().clone().numpy())
    return np.array(path)


runs_2b = {
    r"SGD, $\eta=0.035$": (lambda p: MySGD(p, lr=0.035), "tab:red"),
    r"SGD + momentum, $\eta=0.01, \mu=0.9$": (lambda p: MySGD(p, lr=0.01,
momentum=0.9), "tab:blue"),
    r"Adam, $\eta=0.3$": (lambda p: MyAdam(p, lr=0.3), "tab:green"),
}
paths = {k: trajectory(v[0]) for k, v in runs_2b.items()}

xs, ys = np.meshgrid(np.linspace(-10, 10, 400), np.linspace(-3, 3, 300))
fig, ax = plt.subplots(1, 2, figsize=(16, 5), gridspec_kw=dict(width_ratios=[2,
1]))
ax[0].contour(xs, ys, 0.5 * (xs ** 2 + 50 * ys ** 2), levels=np.logspace(-1, 2.6,
18), cmap="Greys",
              linewidths=0.8)
for k, path in paths.items():
    ax[0].plot(path[:, 0], path[:, 1], "o-", ms=2.5, lw=1, color=runs_2b[k][1],
label=k)
    ax[1].semilogy([f_quad(q) for q in path], color=runs_2b[k][1], label=k)
ax[0].plot(0, 0, "k*", ms=12)
ax[0].set(xlabel="x", ylabel="y", title=r"Trajectories on $f(x,y)=\frac{1}{2}
(x^2+50y^2)$ (100 steps)",
          xlim=(-10, 10), ylim=(-3, 3))
ax[0].legend(loc="upper right")
ax[1].set(xlabel="iteration", ylabel="f (log)", title="Objective vs iteration");
ax[1].grid(alpha=0.3)
plt.tight_layout(); savefig("task2b_trajectories"); plt.show()
display(pd.DataFrame([(k.replace("$", "").replace("\\eta", "lr").replace("\\mu",
"momentum"), *path[-1],
                       f_quad(path[-1])) for k, path in paths.items()],
                     columns=["method", "x after 100 steps", "y after 100 steps",
"f after 100 steps"])
        .style.format({"x after 100 steps": "{:.4f}", "y after 100 steps": "
{:.4f}", "f after 100 steps": "{:.2e}"})
        .hide(axis="index"))
```




| method | x after 100 steps | y after 100 steps | f after 100 steps |
|---|---|---|---|
| SGD, lr=0.035 | -0.2553 | 0.0000 | 3.26e-02 |
| SGD + momentum, lr=0.01, momentum=0.9 | 0.0237 | 0.0043 | 7.51e-04 |
| Adam, lr=0.3 | -0.0580 | -0.0067 | 2.82e-03 |

**Interpretation:** The final objective values are **3.3e-2 for SGD**, **7.5e-4 for momentum** and **2.8e-3 for Adam**.

- **SGD:** each step multiplies $y$ by $1 - 50\eta = -0.75$. So the path jumps from one side of the valley to the other. Each step multiplies $x$ only by 0.965. The steep direction limits the step size, so SGD moves slowly along the flat direction. After 100 steps $x$ is still -0.26.
- **Momentum:** the gradient along $x$ keeps the same sign, so the velocity grows in that direction. The effective step becomes $\eta/(1 - \mu) = 0.1$. Along $y$ the gradient changes sign, so these parts partly cancel. The path overshoots and oscillates, but it reaches the lowest objective.
- **Adam:** Adam scales each coordinate by its own gradient size. So the first steps have about the same size in $x$ and $y$, and the path starts along the diagonal. This removes most of the effect of the different steepness. Near the minimum the step does not shrink with the gradient. So Adam keeps oscillating a little unless the learning rate decays.

Each method uses a different learning rate, so this plot does not rank the optimizers in general.

## 2(c): Learning-rate sensitivity

I train the same CNN (about 0.14M parameters, no batch norm) with each optimizer at learning rates 0.001, 0.01 and 0.1. All nine runs use the same seed and data order, batch size 128, a constant learning rate, 10 epochs, no augmentation and fp32. For each optimizer I choose the learning rate with the best final validation accuracy.

In [11]:
```
class SmallCNN(nn.Module):
    def __init__(self, bn=False, p_drop=0.0, widths=(32, 64, 128)):
        super().__init__()
```

```
        c1, c2, c3 = widths

        def conv(cin, cout):
            layers = [nn.Conv2d(cin, cout, 3, padding=1, bias=not bn)]
            if bn:
                layers.append(nn.BatchNorm2d(cout))
            return layers + [nn.ReLU(inplace=True)]

        self.features = nn.Sequential(
            *conv(3, c1), *conv(c1, c1), nn.MaxPool2d(2), nn.Dropout(p_drop),
            *conv(c1, c2), *conv(c2, c2), nn.MaxPool2d(2), nn.Dropout(p_drop),
            *conv(c2, c3), nn.MaxPool2d(2))
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
nn.Dropout(p_drop), nn.Linear(c3, 10))

    def forward(self, x):
        return self.head(self.features(x))


print("SmallCNN parameters:", n_params(SmallCNN()))

OPTS_2C = {
    "SGD": lambda p, lr: MySGD(p, lr=lr),
    "SGD+momentum": lambda p, lr: MySGD(p, lr=lr, momentum=0.9),
    "Adam": lambda p, lr: MyAdam(p, lr=lr),
}
LRS_2C = [1e-3, 1e-2, 1e-1]


def run_2c(opt_name, lr):
    set_seed()
    model = SmallCNN().to(device)
    opt = OPTS_2C[opt_name](model.parameters(), lr)
    return fit(model, opt, 10, X_tr, y_tr, bs=128, amp=False, tag=f"{opt_name} lr=
{lr:g}")


R2c = {(o, lr): cached(f"task2c_{o}_lr{lr:g}", lambda o=o, lr=lr: run_2c(o, lr))
       for o in OPTS_2C for lr in LRS_2C}
```

```
SmallCNN parameters: 140714
Loaded saved run: task2c_SGD_lr0.001
[SGD lr=0.001] ep   1/10 loss 2.3037 val_acc 0.1022 (8.7s)
[SGD lr=0.001] ep   2/10 loss 2.3035 val_acc 0.1030 (5.0s)
[SGD lr=0.001] ep   3/10 loss 2.3033 val_acc 0.1032 (5.0s)
[SGD lr=0.001] ep   4/10 loss 2.3031 val_acc 0.1028 (5.0s)
[SGD lr=0.001] ep   5/10 loss 2.3029 val_acc 0.1030 (5.0s)
[SGD lr=0.001] ep   6/10 loss 2.3027 val_acc 0.1044 (5.0s)
[SGD lr=0.001] ep   7/10 loss 2.3025 val_acc 0.1032 (5.0s)
[SGD lr=0.001] ep   8/10 loss 2.3023 val_acc 0.1054 (5.0s)
[SGD lr=0.001] ep   9/10 loss 2.3021 val_acc 0.1056 (5.0s)
[SGD lr=0.001] ep  10/10 loss 2.3019 val_acc 0.1058 (5.0s)
Loaded saved run: task2c_SGD_lr0.01
[SGD lr=0.01] ep   1/10 loss 2.3029 val_acc 0.1058 (5.0s)
[SGD lr=0.01] ep   2/10 loss 2.3008 val_acc 0.1370 (5.0s)
[SGD lr=0.01] ep   3/10 loss 2.2963 val_acc 0.1206 (5.0s)
[SGD lr=0.01] ep   4/10 loss 2.2750 val_acc 0.1268 (5.0s)
[SGD lr=0.01] ep   5/10 loss 2.2139 val_acc 0.1756 (5.0s)
[SGD lr=0.01] ep   6/10 loss 2.1316 val_acc 0.2198 (5.0s)
[SGD lr=0.01] ep   7/10 loss 2.0185 val_acc 0.2792 (5.0s)
[SGD lr=0.01] ep   8/10 loss 1.9192 val_acc 0.2996 (5.0s)
[SGD lr=0.01] ep   9/10 loss 1.8372 val_acc 0.3244 (5.0s)
[SGD lr=0.01] ep  10/10 loss 1.7819 val_acc 0.3440 (5.0s)
Loaded saved run: task2c_SGD_lr0.1
[SGD lr=0.1] ep   1/10 loss 2.1774 val_acc 0.2356 (5.0s)
[SGD lr=0.1] ep   2/10 loss 1.8765 val_acc 0.3342 (5.0s)
[SGD lr=0.1] ep   3/10 loss 1.7233 val_acc 0.3806 (5.0s)
[SGD lr=0.1] ep   4/10 loss 1.6233 val_acc 0.3496 (5.0s)
[SGD lr=0.1] ep   5/10 loss 1.5137 val_acc 0.4634 (5.0s)
[SGD lr=0.1] ep   6/10 loss 1.4164 val_acc 0.5302 (5.0s)
[SGD lr=0.1] ep   7/10 loss 1.3140 val_acc 0.4942 (5.0s)
[SGD lr=0.1] ep   8/10 loss 1.2336 val_acc 0.5374 (5.0s)
[SGD lr=0.1] ep   9/10 loss 1.1742 val_acc 0.5412 (5.0s)
[SGD lr=0.1] ep  10/10 loss 1.1000 val_acc 0.5822 (5.0s)
Loaded saved run: task2c_SGD+momentum_lr0.001
[SGD+momentum lr=0.001] ep   1/10 loss 2.3029 val_acc 0.1054 (5.1s)
[SGD+momentum lr=0.001] ep   2/10 loss 2.3010 val_acc 0.1482 (5.1s)
[SGD+momentum lr=0.001] ep   3/10 loss 2.2969 val_acc 0.1320 (5.1s)
[SGD+momentum lr=0.001] ep   4/10 loss 2.2787 val_acc 0.1178 (5.1s)
[SGD+momentum lr=0.001] ep   5/10 loss 2.2247 val_acc 0.1664 (5.1s)
[SGD+momentum lr=0.001] ep   6/10 loss 2.1438 val_acc 0.2280 (5.1s)
[SGD+momentum lr=0.001] ep   7/10 loss 2.0366 val_acc 0.2692 (5.1s)
[SGD+momentum lr=0.001] ep   8/10 loss 1.9301 val_acc 0.3152 (5.1s)
[SGD+momentum lr=0.001] ep   9/10 loss 1.8329 val_acc 0.3378 (5.1s)
[SGD+momentum lr=0.001] ep  10/10 loss 1.7640 val_acc 0.3508 (5.1s)
Loaded saved run: task2c_SGD+momentum_lr0.01
[SGD+momentum lr=0.01] ep   1/10 loss 2.1929 val_acc 0.3038 (5.1s)
[SGD+momentum lr=0.01] ep   2/10 loss 1.7690 val_acc 0.3712 (5.1s)
[SGD+momentum lr=0.01] ep   3/10 loss 1.5860 val_acc 0.3970 (5.1s)
[SGD+momentum lr=0.01] ep   4/10 loss 1.4391 val_acc 0.4782 (5.1s)
[SGD+momentum lr=0.01] ep   5/10 loss 1.2924 val_acc 0.5634 (5.1s)
[SGD+momentum lr=0.01] ep   6/10 loss 1.1789 val_acc 0.6126 (5.1s)
[SGD+momentum lr=0.01] ep   7/10 loss 1.0787 val_acc 0.6110 (5.1s)
[SGD+momentum lr=0.01] ep   8/10 loss 1.0082 val_acc 0.6214 (5.1s)
[SGD+momentum lr=0.01] ep   9/10 loss 0.9316 val_acc 0.6708 (5.1s)
[SGD+momentum lr=0.01] ep  10/10 loss 0.8845 val_acc 0.6850 (5.1s)
```

```
Loaded saved run: task2c_SGD+momentum_lr0.1
[SGD+momentum lr=0.1] ep   1/10 loss 1.9672 val_acc 0.3152 (5.1s)
[SGD+momentum lr=0.1] ep   2/10 loss 1.5052 val_acc 0.4562 (5.1s)
[SGD+momentum lr=0.1] ep   3/10 loss 1.2371 val_acc 0.5884 (5.1s)
[SGD+momentum lr=0.1] ep   4/10 loss 1.0571 val_acc 0.6624 (5.3s)
[SGD+momentum lr=0.1] ep   5/10 loss 0.9384 val_acc 0.6436 (5.1s)
[SGD+momentum lr=0.1] ep   6/10 loss 0.8546 val_acc 0.6644 (5.1s)
[SGD+momentum lr=0.1] ep   7/10 loss 0.7947 val_acc 0.7000 (5.1s)
[SGD+momentum lr=0.1] ep   8/10 loss 0.7425 val_acc 0.7244 (5.1s)
[SGD+momentum lr=0.1] ep   9/10 loss 0.6884 val_acc 0.7138 (5.1s)
[SGD+momentum lr=0.1] ep  10/10 loss 0.6722 val_acc 0.7226 (5.1s)
Loaded saved run: task2c_Adam_lr0.001
[Adam lr=0.001] ep   1/10 loss 1.7235 val_acc 0.4638 (5.2s)
[Adam lr=0.001] ep   2/10 loss 1.3982 val_acc 0.5354 (5.2s)
[Adam lr=0.001] ep   3/10 loss 1.2200 val_acc 0.5616 (5.2s)
[Adam lr=0.001] ep   4/10 loss 1.0920 val_acc 0.6346 (5.2s)
[Adam lr=0.001] ep   5/10 loss 0.9942 val_acc 0.6526 (5.2s)
[Adam lr=0.001] ep   6/10 loss 0.9276 val_acc 0.6778 (5.2s)
[Adam lr=0.001] ep   7/10 loss 0.8704 val_acc 0.6950 (5.2s)
[Adam lr=0.001] ep   8/10 loss 0.8282 val_acc 0.6782 (5.2s)
[Adam lr=0.001] ep   9/10 loss 0.7875 val_acc 0.7012 (5.2s)
[Adam lr=0.001] ep  10/10 loss 0.7550 val_acc 0.7188 (5.2s)
Loaded saved run: task2c_Adam_lr0.01
[Adam lr=0.01] ep   1/10 loss 2.3125 val_acc 0.0934 (5.2s)
[Adam lr=0.01] ep   2/10 loss 2.3032 val_acc 0.0934 (5.2s)
[Adam lr=0.01] ep   3/10 loss 2.3034 val_acc 0.0996 (5.2s)
[Adam lr=0.01] ep   4/10 loss 2.3033 val_acc 0.1008 (5.2s)
[Adam lr=0.01] ep   5/10 loss 2.3032 val_acc 0.1008 (5.2s)
[Adam lr=0.01] ep   6/10 loss 2.3032 val_acc 0.0996 (5.2s)
[Adam lr=0.01] ep   7/10 loss 2.3033 val_acc 0.1008 (5.2s)
[Adam lr=0.01] ep   8/10 loss 2.3033 val_acc 0.1016 (5.2s)
[Adam lr=0.01] ep   9/10 loss 2.3034 val_acc 0.1012 (5.2s)
[Adam lr=0.01] ep  10/10 loss 2.3033 val_acc 0.1012 (5.2s)
Loaded saved run: task2c_Adam_lr0.1
[Adam lr=0.1] ep   1/10 loss 567.2357 val_acc 0.1008 (5.2s)
[Adam lr=0.1] ep   2/10 loss 2.3083 val_acc 0.0934 (5.2s)
[Adam lr=0.1] ep   3/10 loss 2.3093 val_acc 0.1014 (5.2s)
[Adam lr=0.1] ep   4/10 loss 2.3088 val_acc 0.0996 (5.2s)
[Adam lr=0.1] ep   5/10 loss 2.3084 val_acc 0.1008 (5.2s)
[Adam lr=0.1] ep   6/10 loss 2.3091 val_acc 0.0988 (5.2s)
[Adam lr=0.1] ep   7/10 loss 2.3089 val_acc 0.0964 (5.2s)
[Adam lr=0.1] ep   8/10 loss 2.3091 val_acc 0.1016 (5.2s)
[Adam lr=0.1] ep   9/10 loss 2.3086 val_acc 0.1016 (5.2s)
[Adam lr=0.1] ep  10/10 loss 2.3082 val_acc 0.1012 (5.2s)
```

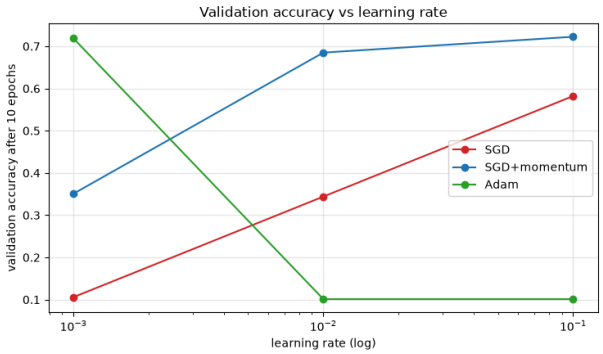
In [12]:

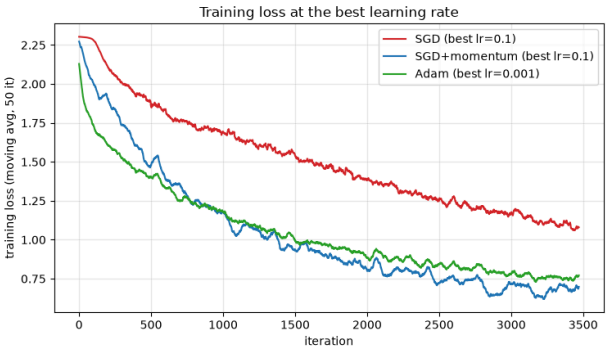
```python
def final_acc(H):
    return H["val_acc"][-1] if math.isfinite(H["train_loss"][-1]) else 0.1


fig, ax = plt.subplots(1, 2, figsize=(15, 4.5))
colors = dict(zip(OPTS_2C, ["tab:red", "tab:blue", "tab:green"]))
for o in OPTS_2C:
    accs = [final_acc(R2c[(o, lr)]) for lr in LRS_2C]
    ax[0].semilogx(LRS_2C, accs, "o-", color=colors[o], label=o)
    best = LRS_2C[int(np.argmax(accs))]
    ax[1].plot(smooth(R2c[(o, best)]["iter_loss"], 50), color=colors[o], label=f"
```

```
{o} (best lr={best:g})")
ax[0].set(xlabel="learning rate (log)", ylabel="validation accuracy after 10
epochs",
          title="Validation accuracy vs learning rate"); ax[0].grid(alpha=0.3);
ax[0].legend()
ax[1].set(xlabel="iteration", ylabel="training loss (moving avg, 50 it)",
          title="Training loss at the best learning rate")
ax[1].grid(alpha=0.3); ax[1].legend()
plt.tight_layout(); savefig("task2c_lr_sensitivity"); plt.show()

t2c = pd.DataFrame({f"lr = {lr:g}": [final_acc(R2c[(o, lr)]) for o in OPTS_2C] for
lr in LRS_2C}, index=list(OPTS_2C))
t2c["best lr"] = [f"{LRS_2C[int(np.argmax(row))]:g}" for row in t2c.values]
t2c["final train loss at best lr"] = [R2c[(o, float(b))]["train_loss"][-1] for o,
b in zip(OPTS_2C, t2c["best lr"])]
print("Final validation accuracy after 10 epochs:")
display(t2c.style.format({c: "{:.2%}" for c in t2c.columns if c.startswith("lr")}
|
                         {"final train loss at best lr": "{:.3f}"}).highlight_max(
    axis=None, subset=[c for c in t2c.columns if c.startswith("lr")],
color="#c6efce"))
```




Final validation accuracy after 10 epochs:

| | lr = 0.001 | lr = 0.01 | lr = 0.1 | best lr | final train loss at best lr |
|---|---|---|---|---|---|
| **SGD** | 10.58% | 34.40% | 58.22% | 0.1 | 1.100 |
| **SGD+momentum** | 35.08% | 68.50% | 72.26% | 0.1 | 0.672 |
| **Adam** | 71.88% | 10.12% | 10.12% | 0.001 | 0.755 |

**Interpretation:**

- **SGD** needs a large learning rate. At 0.001 the loss stays at 2.30 for all 10 epochs, and the accuracy stays at chance level. The accuracy grows with the learning rate. The best result is **58.2%** at 0.1.
- **Momentum** acts like SGD with a 10 times larger learning rate. Momentum at 0.001 gives about the same accuracy as SGD at 0.01 (35.1% and 34.4%). It works well over a wider range: 68.5% at 0.01 and **72.3%** at 0.1. It also reaches the lowest training loss.
- **Adam** lowers the loss fastest at the start. But it works only at 0.001 (**71.9%**). At 0.01 the loss stays at 2.30, and at 0.1 the loss explodes. Adam takes a step of about $\eta$ for every parameter, whatever the gradient size. For this network without batch norm, 0.01 is already too large.

The best learning rates differ by a factor of 100. So each optimizer needs its own tuning. In this experiment momentum gave the best accuracy and was the least sensitive to the learning rate.

# Task 3: Regularization and residual connections

## 3(a): Regularization on 5,000 training images

I use the small CNN from 2(c) and add batch norm after every convolution. I train it on the first 5,000 images of the training split and evaluate it on the full validation split. All runs use the same seed, SGD with momentum 0.9, learning rate 0.05 with cosine decay, batch size 128 and 100 epochs. I measure the training accuracy on the clean (not augmented) training images in eval mode.

- **weight decay:** $5 \times 10^{-4}$
- **dropout:** $p = 0.3$ after each pooling stage and before the classifier
- **flip + crop:** random horizontal flip, then a random 32×32 crop of the image padded by 4 pixels
- **cutout:** one random 16×16 square set to zero (the mean) in each image
- **mixup:** $\tilde{x} = \lambda x_i + (1-\lambda) x_j$ with loss $\lambda \,\mathrm{CE}(y_i) + (1-\lambda)\,\mathrm{CE}(y_j)$, $\lambda \sim \mathrm{Beta}(0.4, 0.4)$
- **all combined:** all of the above together

In [13]:
```
X_small, y_small = X_tr[:5000], y_tr[:5000]
print("5k subset class counts:", torch.bincount(y_small.cpu()).tolist())

VARIANTS_3A = {
    "baseline": dict(),
    "weight decay": dict(wd=5e-4),
    "dropout": dict(p_drop=0.3),
    "flip+crop": dict(aug=flip_crop),
    "cutout": dict(aug=cutout),
    "mixup": dict(mixup_alpha=0.4),
    "all combined": dict(wd=5e-4, p_drop=0.3, aug=compose(flip_crop, cutout),
mixup_alpha=0.4),
}


def run_3a(name, n_epochs=100):
    cfg = VARIANTS_3A[name]
    set_seed()
    model = SmallCNN(bn=True, p_drop=cfg.get("p_drop", 0.0)).to(device)
    opt = MySGD(model.parameters(), lr=0.05, momentum=0.9,
weight_decay=cfg.get("wd", 0.0))
    steps = n_epochs * math.ceil(len(y_small) / 128)
    return fit(model, opt, n_epochs, X_small, y_small, bs=128, aug=cfg.get("aug"),
               mixup_alpha=cfg.get("mixup_alpha", 0.0), sched=cosine_warmup(opt,
steps), eval_train=True,
               tag=name)
```

```
R3a = {k: cached(f"task3a_{k.replace(' ', '_').replace('+', '_')}", lambda k=k:
run_3a(k, 100))
       for k in VARIANTS_3A}
```

```
5k subset class counts: [511, 510, 506, 509, 502, 472, 500, 470, 500, 520]
Loaded saved run: task3a_baseline
[baseline] ep   1/100 loss 1.8772 val_acc 0.2764 train_acc 0.2906 (15.6s)
[baseline] ep  10/100 loss 0.9129 val_acc 0.4872 train_acc 0.5562 (3.1s)
[baseline] ep  20/100 loss 0.3398 val_acc 0.6164 train_acc 0.8358 (3.1s)
[baseline] ep  30/100 loss 0.1767 val_acc 0.6224 train_acc 0.9300 (3.1s)
[baseline] ep  40/100 loss 0.1188 val_acc 0.6326 train_acc 0.9600 (3.1s)
[baseline] ep  50/100 loss 0.0296 val_acc 0.6306 train_acc 0.9606 (3.1s)
[baseline] ep  60/100 loss 0.0096 val_acc 0.6826 train_acc 1.0000 (3.1s)
[baseline] ep  70/100 loss 0.0084 val_acc 0.6806 train_acc 1.0000 (3.1s)
[baseline] ep  80/100 loss 0.0098 val_acc 0.6770 train_acc 1.0000 (3.1s)
[baseline] ep  90/100 loss 0.0089 val_acc 0.6832 train_acc 1.0000 (3.1s)
[baseline] ep 100/100 loss 0.0088 val_acc 0.6818 train_acc 1.0000 (3.1s)
Loaded saved run: task3a_weight_decay
[weight decay] ep   1/100 loss 1.8744 val_acc 0.1948 train_acc 0.2076 (3.1s)
[weight decay] ep  10/100 loss 0.9319 val_acc 0.4756 train_acc 0.5416 (3.1s)
[weight decay] ep  20/100 loss 0.3781 val_acc 0.5636 train_acc 0.7862 (3.1s)
[weight decay] ep  30/100 loss 0.1919 val_acc 0.6470 train_acc 0.9520 (3.1s)
[weight decay] ep  40/100 loss 0.1171 val_acc 0.6270 train_acc 0.9568 (3.1s)
[weight decay] ep  50/100 loss 0.0207 val_acc 0.6822 train_acc 1.0000 (3.1s)
[weight decay] ep  60/100 loss 0.0100 val_acc 0.6778 train_acc 1.0000 (3.1s)
[weight decay] ep  70/100 loss 0.0168 val_acc 0.6744 train_acc 1.0000 (3.1s)
[weight decay] ep  80/100 loss 0.0066 val_acc 0.6792 train_acc 1.0000 (3.1s)
[weight decay] ep  90/100 loss 0.0113 val_acc 0.6802 train_acc 1.0000 (3.1s)
[weight decay] ep 100/100 loss 0.0129 val_acc 0.6790 train_acc 1.0000 (3.1s)
Loaded saved run: task3a_dropout
[dropout] ep   1/100 loss 2.0229 val_acc 0.2190 train_acc 0.2238 (3.1s)
[dropout] ep  10/100 loss 1.3257 val_acc 0.5016 train_acc 0.5252 (3.1s)
[dropout] ep  20/100 loss 1.0154 val_acc 0.5788 train_acc 0.6476 (3.1s)
[dropout] ep  30/100 loss 0.8561 val_acc 0.6342 train_acc 0.7668 (3.1s)
[dropout] ep  40/100 loss 0.7056 val_acc 0.6480 train_acc 0.8058 (3.1s)
[dropout] ep  50/100 loss 0.5596 val_acc 0.6580 train_acc 0.8500 (3.1s)
[dropout] ep  60/100 loss 0.4986 val_acc 0.6790 train_acc 0.9154 (3.1s)
[dropout] ep  70/100 loss 0.3921 val_acc 0.6812 train_acc 0.9454 (3.1s)
[dropout] ep  80/100 loss 0.3359 val_acc 0.6958 train_acc 0.9700 (3.1s)
[dropout] ep  90/100 loss 0.2946 val_acc 0.6996 train_acc 0.9810 (3.1s)
[dropout] ep 100/100 loss 0.2738 val_acc 0.7010 train_acc 0.9820 (3.1s)
Loaded saved run: task3a_flip_crop
[flip+crop] ep   1/100 loss 1.9330 val_acc 0.2236 train_acc 0.2352 (3.1s)
[flip+crop] ep  10/100 loss 1.1583 val_acc 0.4868 train_acc 0.5142 (3.1s)
[flip+crop] ep  20/100 loss 0.8471 val_acc 0.6330 train_acc 0.7096 (3.1s)
[flip+crop] ep  30/100 loss 0.7165 val_acc 0.6234 train_acc 0.7068 (3.1s)
[flip+crop] ep  40/100 loss 0.5196 val_acc 0.6914 train_acc 0.8206 (3.1s)
[flip+crop] ep  50/100 loss 0.4720 val_acc 0.7058 train_acc 0.8614 (3.1s)
[flip+crop] ep  60/100 loss 0.3209 val_acc 0.7002 train_acc 0.8782 (3.1s)
[flip+crop] ep  70/100 loss 0.2544 val_acc 0.7282 train_acc 0.9254 (3.1s)
[flip+crop] ep  80/100 loss 0.1868 val_acc 0.7378 train_acc 0.9622 (3.1s)
[flip+crop] ep  90/100 loss 0.1660 val_acc 0.7422 train_acc 0.9678 (3.1s)
[flip+crop] ep 100/100 loss 0.1664 val_acc 0.7394 train_acc 0.9680 (3.1s)
Loaded saved run: task3a_cutout
[cutout] ep   1/100 loss 1.9516 val_acc 0.2094 train_acc 0.2240 (3.1s)
[cutout] ep  10/100 loss 1.1588 val_acc 0.5576 train_acc 0.6256 (3.1s)
[cutout] ep  20/100 loss 0.9291 val_acc 0.5586 train_acc 0.6602 (3.1s)
[cutout] ep  30/100 loss 0.6297 val_acc 0.5938 train_acc 0.7548 (3.1s)
[cutout] ep  40/100 loss 0.4569 val_acc 0.5670 train_acc 0.7800 (3.1s)
[cutout] ep  50/100 loss 0.3436 val_acc 0.6758 train_acc 0.9814 (3.1s)
```

```
[cutout] ep  60/100 loss 0.2255 val_acc 0.6898 train_acc 0.9946 (3.1s)
[cutout] ep  70/100 loss 0.1212 val_acc 0.6932 train_acc 0.9984 (3.1s)
[cutout] ep  80/100 loss 0.0909 val_acc 0.7044 train_acc 0.9998 (3.1s)
[cutout] ep  90/100 loss 0.0732 val_acc 0.7068 train_acc 1.0000 (3.1s)
[cutout] ep 100/100 loss 0.0679 val_acc 0.7108 train_acc 1.0000 (3.1s)
Loaded saved run: task3a_mixup
[mixup] ep   1/100 loss 2.0170 val_acc 0.2322 train_acc 0.2416 (3.1s)
[mixup] ep  10/100 loss 1.5483 val_acc 0.5348 train_acc 0.5964 (3.1s)
[mixup] ep  20/100 loss 1.1864 val_acc 0.5788 train_acc 0.7316 (3.1s)
[mixup] ep  30/100 loss 0.8939 val_acc 0.5618 train_acc 0.7788 (3.1s)
[mixup] ep  40/100 loss 0.8063 val_acc 0.6450 train_acc 0.9682 (3.1s)
[mixup] ep  50/100 loss 0.7492 val_acc 0.6114 train_acc 0.9738 (3.1s)
[mixup] ep  60/100 loss 0.7123 val_acc 0.6690 train_acc 0.9996 (3.1s)
[mixup] ep  70/100 loss 0.7283 val_acc 0.6740 train_acc 1.0000 (3.1s)
[mixup] ep  80/100 loss 0.4890 val_acc 0.6896 train_acc 1.0000 (3.1s)
[mixup] ep  90/100 loss 0.6846 val_acc 0.6816 train_acc 1.0000 (3.1s)
[mixup] ep 100/100 loss 0.6056 val_acc 0.6860 train_acc 1.0000 (3.1s)
Loaded saved run: task3a_all_combined
[all combined] ep   1/100 loss 2.1918 val_acc 0.1856 train_acc 0.1888 (3.1s)
[all combined] ep  10/100 loss 1.8722 val_acc 0.4184 train_acc 0.4202 (3.1s)
[all combined] ep  20/100 loss 1.7135 val_acc 0.4970 train_acc 0.5172 (3.1s)
[all combined] ep  30/100 loss 1.5614 val_acc 0.5512 train_acc 0.5838 (3.1s)
[all combined] ep  40/100 loss 1.5189 val_acc 0.5982 train_acc 0.6580 (3.2s)
[all combined] ep  50/100 loss 1.4852 val_acc 0.6178 train_acc 0.6722 (3.2s)
[all combined] ep  60/100 loss 1.4528 val_acc 0.6418 train_acc 0.7180 (3.2s)
[all combined] ep  70/100 loss 1.4433 val_acc 0.6398 train_acc 0.7112 (3.2s)
[all combined] ep  80/100 loss 1.2910 val_acc 0.6746 train_acc 0.7578 (3.2s)
[all combined] ep  90/100 loss 1.4022 val_acc 0.6672 train_acc 0.7590 (3.2s)
[all combined] ep 100/100 loss 1.3265 val_acc 0.6700 train_acc 0.7618 (3.2s)
```

In [14]:

```
t3a = pd.DataFrame({k: {"train acc": H["train_acc"][-1], "val acc": H["val_acc"]
[-1],
                        "gap (train - val)": H["train_acc"][-1] - H["val_acc"]
[-1],
                        "best val acc": max(H["val_acc"]), "final val loss":
H["val_loss"][-1]}
                    for k, H in R3a.items()}).T
print("Final epoch (100) on the 5k-image training subset:")
display(t3a.style.format({c: "{:.2%}" for c in t3a.columns if c != "final val
loss"} | {"final val loss": "{:.3f}"})
        .highlight_max(subset=["val acc"], color="#c6efce").highlight_min(subset=
["gap (train - val)"], color="#c6efce"))

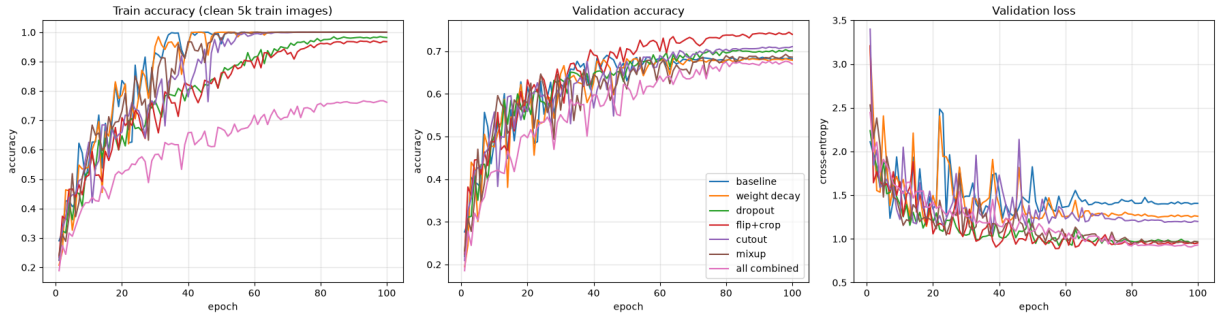
fig, axes = plt.subplots(1, 3, figsize=(18, 4.8))
cmap = plt.get_cmap("tab10")
for i, (k, H) in enumerate(R3a.items()):
    ep = np.arange(1, len(H["val_acc"]) + 1)
    axes[0].plot(ep, H["train_acc"], color=cmap(i), label=k)
    axes[1].plot(ep, H["val_acc"], color=cmap(i), label=k)
    axes[2].plot(ep, H["val_loss"], color=cmap(i), label=k)
axes[0].set(xlabel="epoch", ylabel="accuracy", title="Train accuracy (clean 5k
train images)")
axes[1].set(xlabel="epoch", ylabel="accuracy", title="Validation accuracy")
axes[2].set(xlabel="epoch", ylabel="cross-entropy", title="Validation loss", ylim=
(0.5, 3.5))
for a in axes:
```

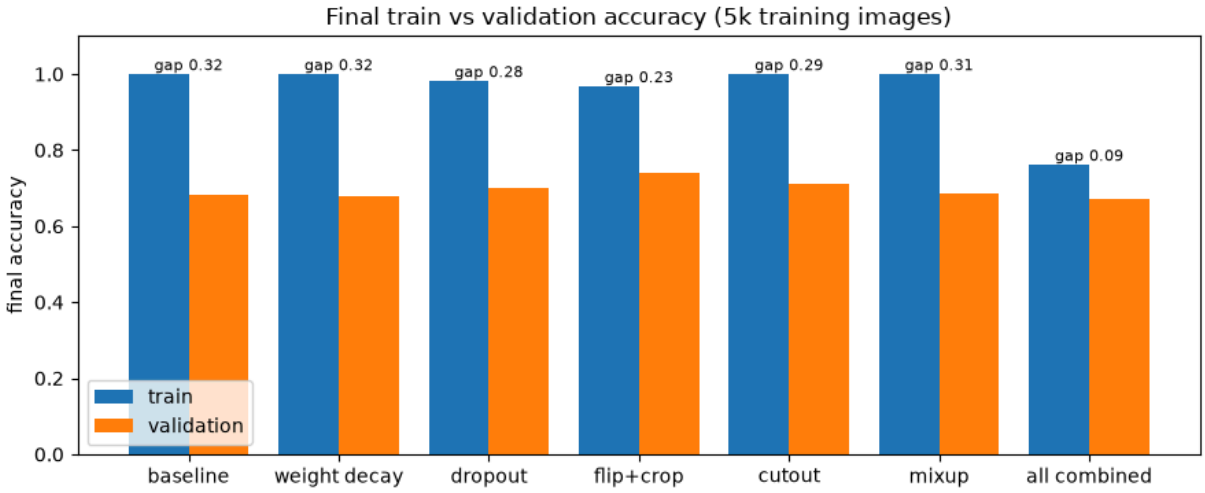
```
    a.grid(alpha=0.3)
axes[1].legend(loc="lower right")
plt.tight_layout(); savefig("task3a_regularization"); plt.show()

fig, ax = plt.subplots(figsize=(9, 3.8))
names = list(R3a)
tr = [R3a[k]["train_acc"][-1] for k in names]; va = [R3a[k]["val_acc"][-1] for k
in names]
xpos = np.arange(len(names))
ax.bar(xpos - 0.2, tr, 0.4, label="train"); ax.bar(xpos + 0.2, va, 0.4,
label="validation")
for x_, t_, v_ in zip(xpos, tr, va):
    ax.text(x_, max(t_, v_) + 0.01, f"gap {t_ - v_:.2f}", ha="center", fontsize=8)
ax.set_xticks(xpos); ax.set_xticklabels(names); ax.set_ylim(0, 1.1);
ax.set_ylabel("final accuracy")
ax.set_title("Final train vs validation accuracy (5k training images)");
ax.legend(loc="lower left")
plt.tight_layout(); savefig("task3a_gap"); plt.show()
```

Final epoch (100) on the 5k-image training subset:

| | train acc | val acc | gap (train - val) | best val acc | final val loss |
|---|---|---|---|---|---|
| **baseline** | 100.00% | 68.18% | 31.82% | 69.04% | 1.406 |
| **weight decay** | 100.00% | 67.90% | 32.10% | 68.44% | 1.259 |
| **dropout** | 98.20% | 70.10% | 28.10% | 70.36% | 0.967 |
| **flip+crop** | 96.80% | 73.94% | 22.86% | 74.44% | 0.953 |
| **cutout** | 100.00% | 71.08% | 28.92% | 71.08% | 1.199 |
| **mixup** | 100.00% | 68.60% | 31.40% | 69.26% | 0.965 |
| **all combined** | 76.18% | 67.00% | 9.18% | 67.70% | 0.929 |






**Interpretation:** The baseline fits the 5,000 training images perfectly (100% training accuracy). It reaches **68.2%** validation accuracy, so the gap is **31.8 points**.

- **Flip + crop** works best on its own (**73.9%**, gap 22.9). A flipped or slightly shifted image still shows the same class. So this works like adding new training images.
- **Cutout** (71.1%) and **dropout** (70.1%) also help. Cutout forces the model to use more parts of the object. Dropout stops the model from relying on a few features. Only dropout and flip + crop keep the training accuracy below 100%.
- **Weight decay** does not change the accuracy (67.9%). Batch norm follows every convolution, so the output does not depend on the scale of the convolution weights. Weight decay then mainly changes the effective learning rate. It does lower the validation loss from 1.41 to 1.26.
- **Mixup** gives about the same accuracy as the baseline (68.6%). But it lowers the validation loss from 1.41 to 0.97, so the predictions are less overconfident.
- **All combined** gives the smallest gap (**9.2 points**) and the lowest validation loss. But the training accuracy reaches only 76.2%, and the validation accuracy (67.0%) is lower than with flip + crop alone. Together the methods regularize too much for this small model and 100 epochs. The curves are still rising at the end of training.

Combining all methods reduces the gap, but it does not give the best accuracy here.

## 3(b): Plain and residual networks of depth 6n + 2

I follow He et al. (2016). The network starts with a 3×3 convolution with 16 channels. Then come three stages of $n$ blocks with 16, 32 and 64 channels. Each block has two 3×3 convolutions with batch norm. The first convolution of stages 2 and 3 has stride 2. Global average pooling and a linear layer come at the end. Depths 8, 20 and 56 use $n$ = 1, 3 and 9. All convolutions use Kaiming-normal initialization.

The two families differ only in the shortcut. A residual block computes $\mathrm{ReLU}(F(x) + x)$, and a plain block computes $\mathrm{ReLU}(F(x))$. When the shape changes, the shortcut takes every second pixel and pads the extra channels with zeros (option A). This shortcut has no

parameters, so a plain and a residual network of the same depth have exactly the same parameters.

I train all six networks the same way: SGD with momentum 0.9 and weight decay $10^{-4}$, batch size 128, learning rate 0.1 with one epoch of warm-up and cosine decay, flip + crop augmentation, and 20 epochs on the 45,000 training images.

In [15]:
```
class ShortcutA(nn.Module):
    """Identity shortcut; on a shape change, subsample spatially and zero-pad the
new channels."""
    def __init__(self, c_in, c_out, stride):
        super().__init__()
        self.stride, self.pad = stride, c_out - c_in

    def forward(self, x):
        if self.stride > 1:
            x = x[:, :, ::self.stride, ::self.stride]
        if self.pad > 0:
            x = F.pad(x, (0, 0, 0, 0, self.pad // 2, self.pad - self.pad // 2))
        return x


class ShortcutB(nn.Sequential):
    """Projection shortcut (1x1 conv + BN) on a shape change, identity
otherwise."""
    def __init__(self, c_in, c_out, stride):
        if stride == 1 and c_in == c_out:
            super().__init__()
        else:
            super().__init__(nn.Conv2d(c_in, c_out, 1, stride, bias=False),
nn.BatchNorm2d(c_out))


class BasicBlock(nn.Module):
    def __init__(self, c_in, c_out, stride, residual=True, shortcut="A"):
        super().__init__()
        self.conv1 = nn.Conv2d(c_in, c_out, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(c_out)
        self.conv2 = nn.Conv2d(c_out, c_out, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(c_out)
        self.residual = residual
        if residual:
            self.shortcut = (ShortcutA if shortcut == "A" else ShortcutB)(c_in,
c_out, stride)

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        if self.residual:
            out = out + self.shortcut(x)
        return F.relu(out)


class CifarNet(nn.Module):
```

```
    """CIFAR ResNet / plain net: stem + 3 stages + GAP + FC. blocks=(n, n, n)
gives depth 6n+2."""
    def __init__(self, blocks=(3, 3, 3), widths=(16, 32, 64), residual=True,
shortcut="A", zero_init_last_bn=False):
        super().__init__()
        self.stem = nn.Sequential(nn.Conv2d(3, widths[0], 3, 1, 1, bias=False),
nn.BatchNorm2d(widths[0]),
                                  nn.ReLU(inplace=True))
        layers, c_in = [], widths[0]
        for s, (n, w) in enumerate(zip(blocks, widths)):
            for b in range(n):
                layers.append(BasicBlock(c_in, w, 2 if (s > 0 and b == 0) else 1,
residual, shortcut))
                c_in = w
        self.stages = nn.Sequential(*layers)
        self.fc = nn.Linear(c_in, 10)
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode="fan_out",
nonlinearity="relu")
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.ones_(m.weight); nn.init.zeros_(m.bias)
            elif isinstance(m, nn.Linear):
                nn.init.kaiming_normal_(m.weight); nn.init.zeros_(m.bias)
        if zero_init_last_bn and residual:
            for m in self.modules():
                if isinstance(m, BasicBlock):
                    nn.init.zeros_(m.bn2.weight)

    def forward(self, x):
        x = self.stages(self.stem(x))
        return self.fc(torch.flatten(F.adaptive_avg_pool2d(x, 1), 1))


def depth_net(depth, residual):
    n = (depth - 2) // 6
    return CifarNet(blocks=(n, n, n), residual=residual)


for d in (8, 20, 56):
    print(f"depth {d:2d}: params plain {n_params(depth_net(d, False)):,}  residual
{n_params(depth_net(d, True)):,}")
```

```
depth  8: params plain 75,290  residual 75,290
depth 20: params plain 269,722  residual 269,722
depth 56: params plain 853,018  residual 853,018
```

In [16]:

```
def run_3b(depth, residual, n_epochs=20):
    set_seed()
    model = depth_net(depth, residual).to(device)
    opt = MySGD(model.parameters(), lr=0.1, momentum=0.9, weight_decay=1e-4)
    spe = math.ceil(len(y_tr) / 128)
    return fit(model, opt, n_epochs, X_tr, y_tr, bs=128, aug=flip_crop,
               sched=cosine_warmup(opt, n_epochs * spe, warmup_steps=spe),
               tag=f"{'res' if residual else 'plain'}-{depth}")
```

```
R3b = {(d, r): cached(f"task3b_{'res' if r else 'plain'}{d}", lambda d=d, r=r:
run_3b(d, r, 20))
       for d in (8, 20, 56) for r in (False, True)}
```

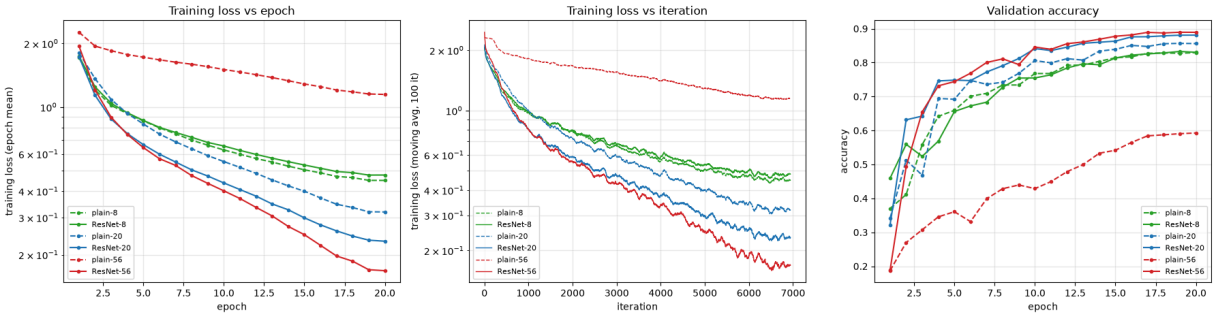
```
Loaded saved run: task3b_plain8
[plain-8] ep   1/20 loss 1.7175 val_acc 0.3700 (6.6s)
[plain-8] ep   2/20 loss 1.2060 val_acc 0.4112 (4.9s)
[plain-8] ep   4/20 loss 0.9305 val_acc 0.6416 (4.9s)
[plain-8] ep   6/20 loss 0.7973 val_acc 0.7008 (4.9s)
[plain-8] ep   8/20 loss 0.6970 val_acc 0.7340 (5.0s)
[plain-8] ep  10/20 loss 0.6266 val_acc 0.7678 (4.9s)
[plain-8] ep  12/20 loss 0.5706 val_acc 0.7922 (4.9s)
[plain-8] ep  14/20 loss 0.5251 val_acc 0.8038 (4.9s)
[plain-8] ep  16/20 loss 0.4880 val_acc 0.8170 (4.9s)
[plain-8] ep  18/20 loss 0.4634 val_acc 0.8284 (4.9s)
[plain-8] ep  20/20 loss 0.4495 val_acc 0.8294 (4.9s)
Loaded saved run: task3b_res8
[res-8] ep   1/20 loss 1.7412 val_acc 0.4592 (5.3s)
[res-8] ep   2/20 loss 1.2452 val_acc 0.5592 (5.3s)
[res-8] ep   4/20 loss 0.9396 val_acc 0.5686 (5.3s)
[res-8] ep   6/20 loss 0.8013 val_acc 0.6724 (5.3s)
[res-8] ep   8/20 loss 0.7190 val_acc 0.7274 (5.3s)
[res-8] ep  10/20 loss 0.6539 val_acc 0.7548 (5.2s)
[res-8] ep  12/20 loss 0.5968 val_acc 0.7842 (5.3s)
[res-8] ep  14/20 loss 0.5513 val_acc 0.7936 (5.2s)
[res-8] ep  16/20 loss 0.5133 val_acc 0.8220 (5.3s)
[res-8] ep  18/20 loss 0.4891 val_acc 0.8286 (5.3s)
[res-8] ep  20/20 loss 0.4767 val_acc 0.8304 (5.3s)
Loaded saved run: task3b_plain20
[plain-20] ep   1/20 loss 1.8075 val_acc 0.3412 (13.3s)
[plain-20] ep   2/20 loss 1.3610 val_acc 0.5128 (13.3s)
[plain-20] ep   4/20 loss 0.9322 val_acc 0.6944 (13.4s)
[plain-20] ep   6/20 loss 0.7447 val_acc 0.7476 (13.4s)
[plain-20] ep   8/20 loss 0.6346 val_acc 0.7422 (13.4s)
[plain-20] ep  10/20 loss 0.5520 val_acc 0.8060 (13.5s)
[plain-20] ep  12/20 loss 0.4856 val_acc 0.8116 (13.6s)
[plain-20] ep  14/20 loss 0.4236 val_acc 0.8338 (13.4s)
[plain-20] ep  16/20 loss 0.3715 val_acc 0.8506 (13.5s)
[plain-20] ep  18/20 loss 0.3345 val_acc 0.8556 (13.4s)
[plain-20] ep  20/20 loss 0.3187 val_acc 0.8560 (13.3s)
Loaded saved run: task3b_res20
[res-20] ep   1/20 loss 1.7330 val_acc 0.3216 (14.1s)
[res-20] ep   2/20 loss 1.1359 val_acc 0.6314 (14.1s)
[res-20] ep   4/20 loss 0.7490 val_acc 0.7460 (14.1s)
[res-20] ep   6/20 loss 0.5984 val_acc 0.7470 (14.1s)
[res-20] ep   8/20 loss 0.5044 val_acc 0.7910 (14.1s)
[res-20] ep  10/20 loss 0.4373 val_acc 0.8412 (14.1s)
[res-20] ep  12/20 loss 0.3787 val_acc 0.8456 (14.1s)
[res-20] ep  14/20 loss 0.3265 val_acc 0.8600 (14.2s)
[res-20] ep  16/20 loss 0.2770 val_acc 0.8758 (14.1s)
[res-20] ep  18/20 loss 0.2451 val_acc 0.8788 (14.2s)
[res-20] ep  20/20 loss 0.2317 val_acc 0.8810 (14.1s)
Loaded saved run: task3b_plain56
[plain-56] ep   1/20 loss 2.2587 val_acc 0.1870 (39.1s)
[plain-56] ep   2/20 loss 1.9416 val_acc 0.2698 (39.1s)
[plain-56] ep   4/20 loss 1.7712 val_acc 0.3456 (39.1s)
[plain-56] ep   6/20 loss 1.6722 val_acc 0.3314 (39.1s)
[plain-56] ep   8/20 loss 1.5948 val_acc 0.4286 (39.4s)
[plain-56] ep  10/20 loss 1.5054 val_acc 0.4288 (39.4s)
[plain-56] ep  12/20 loss 1.4228 val_acc 0.4788 (39.3s)
```

```
[plain-56] ep  14/20 loss 1.3320 val_acc 0.5330 (39.3s)
[plain-56] ep  16/20 loss 1.2483 val_acc 0.5648 (39.5s)
[plain-56] ep  18/20 loss 1.1796 val_acc 0.5872 (39.3s)
[plain-56] ep  20/20 loss 1.1468 val_acc 0.5926 (39.2s)
Loaded saved run: task3b_res56
[res-56] ep   1/20 loss 1.9436 val_acc 0.1884 (41.3s)
[res-56] ep   2/20 loss 1.2051 val_acc 0.4934 (41.0s)
[res-56] ep   4/20 loss 0.7420 val_acc 0.7316 (41.2s)
[res-56] ep   6/20 loss 0.5695 val_acc 0.7688 (41.1s)
[res-56] ep   8/20 loss 0.4738 val_acc 0.8110 (40.9s)
[res-56] ep  10/20 loss 0.4012 val_acc 0.8458 (41.0s)
[res-56] ep  12/20 loss 0.3351 val_acc 0.8558 (41.0s)
[res-56] ep  14/20 loss 0.2723 val_acc 0.8688 (41.0s)
[res-56] ep  16/20 loss 0.2216 val_acc 0.8816 (41.0s)
[res-56] ep  18/20 loss 0.1864 val_acc 0.8872 (41.0s)
[res-56] ep  20/20 loss 0.1682 val_acc 0.8892 (41.0s)
```

In [17]:

```
fig, ax = plt.subplots(1, 3, figsize=(18, 4.8))
dcol = {8: "tab:green", 20: "tab:blue", 56: "tab:red"}
for (d, r), H in R3b.items():
    ep = np.arange(1, len(H["train_loss"]) + 1)
    style = dict(color=dcol[d], ls="-" if r else "--", label=f"{'ResNet' if r else
'plain'}-{d}")
    ax[0].plot(ep, H["train_loss"], marker="o", ms=3, **style)
    ax[1].plot(smooth(H["iter_loss"], 100), lw=1, **style)
    ax[2].plot(ep, H["val_acc"], marker="o", ms=3, **style)
ax[0].set(xlabel="epoch", ylabel="training loss (epoch mean)", title="Training
loss vs epoch", yscale="log")
ax[1].set(xlabel="iteration", ylabel="training loss (moving avg, 100 it)",
title="Training loss vs iteration",
          yscale="log")
ax[2].set(xlabel="epoch", ylabel="accuracy", title="Validation accuracy")
for a in ax:
    a.grid(alpha=0.3, which="both"); a.legend(fontsize=8)
plt.tight_layout(); savefig("task3b_plain_vs_residual"); plt.show()

t3b = pd.DataFrame([{"depth": d, "parameters": n_params(depth_net(d, True)),
                     "plain: train loss": R3b[(d, False)]["train_loss"][-1],
                     "residual: train loss": R3b[(d, True)]["train_loss"][-1],
                     "plain: val acc": R3b[(d, False)]["val_acc"][-1],
                     "residual: val acc": R3b[(d, True)]["val_acc"][-1],
                     "plain: s/epoch": np.mean(R3b[(d, False)]["time"]),
                     "residual: s/epoch": np.mean(R3b[(d, True)]["time"])} for d
in (8, 20, 56)]).set_index("depth")
print("Final epoch (20):")
display(t3b.style.format({"parameters": "{:,}"} | {c: "{:.4f}" for c in
t3b.columns if "loss" in c}
                         | {c: "{:.2%}" for c in t3b.columns if "acc" in c}
                         | {c: "{:.1f}" for c in t3b.columns if "s/epoch" in c}))
```




Final epoch (20):

| depth | parameters | plain: train loss | residual: train loss | plain: val acc | residual: val acc | plain: s/epoch | residual: s/epoch |
|---|---|---|---|---|---|---|---|
| **8** | 75,290 | 0.4495 | 0.4767 | 82.94% | 83.04% | 5.0 | 5.3 |
| **20** | 269,722 | 0.3187 | 0.2317 | 85.60% | 88.10% | 13.4 | 14.1 |
| **56** | 853,018 | 1.1468 | 0.1682 | 59.26% | 88.92% | 39.3 | 41.1 |

**Interpretation:**

- **Plain networks get worse with depth.** Plain-56 ends with a training loss of **1.15**. This is much higher than plain-20 (0.32) and plain-8 (0.45). Its validation accuracy is only 59.3%. The *training* loss is higher, so this is an optimization problem and not overfitting. A 56-layer network can represent anything the 8-layer network can, but SGD does not find such a solution. He et al. call this the degradation problem.
- **Residual networks improve with depth.** The training loss drops from 0.48 to 0.23 to **0.17**. The validation accuracy rises from 83.0% to 88.1% to **88.9%**. Each block only learns a correction $F(x)$ that it adds to $x$. So a block can easily act as the identity. The shortcuts also carry the gradient straight to the early layers.
- At depth 8 the shortcut makes no difference. The gap grows with depth: 2.5 points at depth 20 and 29.7 points at depth 56.
- Deeper networks need more compute. ResNet-56 takes about 8 times longer per epoch than ResNet-8.

# Task 4 (optional): Best CNN with at most 500k parameters and 20 epochs

**Design.** Task 3(b) showed that residual networks train well. With only 20 epochs, a shallow and wide network uses the budget better than a deep and thin one. My model is a ResNet with 2 blocks per stage (14 layers) and widths 26, 52 and 104. It uses 1×1 convolution shortcuts where the shape changes. The last batch-norm scale in each block starts at zero, so each block starts as the identity. The model has 460,548 parameters.

Training: SGD with Nesterov momentum 0.9, batch size 128, peak learning rate 0.1 with one epoch of warm-up and cosine decay, weight decay $5 \times 10^{-4}$, label smoothing 0.1, flip +

crop and cutout (16×16), and 20 epochs on the 45,000 training images.

**Ablation.** Each variant changes one design choice. All variants use the same seed and budget. I pick the variant with the best final validation accuracy as the final model. Only this model sees the test set.

In [18]:
```
T4_ARCH = dict(blocks=(2, 2, 2), widths=(26, 52, 104), shortcut="B",
zero_init_last_bn=True)
T4_PARAMS = n_params(CifarNet(**T4_ARCH))
print(f"Task 4 model parameters: {T4_PARAMS:,}")
assert T4_PARAMS <= 500_000

VARIANTS_4 = {
    "full model": dict(),
    "no shortcut (plain)": dict(residual=False),
    "no augmentation": dict(aug=None),
    "no cutout (flip+crop only)": dict(aug=flip_crop),
    "no weight decay": dict(wd=0.0),
    "no label smoothing": dict(ls=0.0),
    "AdamW instead of SGD": dict(opt="adamw"),
}


def run_4(name, n_epochs=20):
    cfg = VARIANTS_4[name]
    set_seed()
    model = CifarNet(**{**T4_ARCH, "residual": cfg.get("residual",
True)}).to(device)
    wd = cfg.get("wd", 5e-4)
    if cfg.get("opt") == "adamw":
        opt = torch.optim.AdamW(model.parameters(), lr=2e-3, weight_decay=5e-2)
    else:
        opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9,
nesterov=True, weight_decay=wd)
    spe = math.ceil(len(y_tr) / 128)
    H = fit(model, opt, n_epochs, X_tr, y_tr, bs=128, aug=cfg.get("aug",
compose(flip_crop, cutout)),
            label_smoothing=cfg.get("ls", 0.1), sched=cosine_warmup(opt, n_epochs
* spe, warmup_steps=spe),
            tag=name)
    H["params"] = n_params(model)
    torch.save(model.state_dict(), f"{CKPT}/task4_{name.replace(' ', '_')}.pt")
    return H


R4 = {k: cached(f"task4_{k.replace(' ', '_').replace('(', '').replace(')',
'').replace('+', '_')}",
                lambda k=k: run_4(k, 20)) for k in VARIANTS_4}
```

```
Task 4 model parameters: 460,548
Loaded saved run: task4_full_model
[full model] ep   1/20 loss 1.9915 val_acc 0.4166 (17.3s)
[full model] ep   2/20 loss 1.6357 val_acc 0.4784 (15.0s)
[full model] ep   4/20 loss 1.3462 val_acc 0.6280 (15.0s)
[full model] ep   6/20 loss 1.2196 val_acc 0.6546 (15.0s)
[full model] ep   8/20 loss 1.1243 val_acc 0.7032 (15.0s)
[full model] ep  10/20 loss 1.0550 val_acc 0.7668 (15.0s)
[full model] ep  12/20 loss 0.9978 val_acc 0.7980 (15.0s)
[full model] ep  14/20 loss 0.9393 val_acc 0.8452 (15.0s)
[full model] ep  16/20 loss 0.8796 val_acc 0.8670 (15.0s)
[full model] ep  18/20 loss 0.8405 val_acc 0.8882 (15.0s)
[full model] ep  20/20 loss 0.8228 val_acc 0.8896 (15.0s)
Loaded saved run: task4_no_shortcut_plain
[no shortcut (plain)] ep   1/20 loss 1.8539 val_acc 0.4826 (13.8s)
[no shortcut (plain)] ep   2/20 loss 1.4760 val_acc 0.6194 (13.8s)
[no shortcut (plain)] ep   4/20 loss 1.2455 val_acc 0.7080 (13.8s)
[no shortcut (plain)] ep   6/20 loss 1.1478 val_acc 0.7112 (13.8s)
[no shortcut (plain)] ep   8/20 loss 1.0859 val_acc 0.6954 (13.8s)
[no shortcut (plain)] ep  10/20 loss 1.0365 val_acc 0.7900 (13.8s)
[no shortcut (plain)] ep  12/20 loss 0.9869 val_acc 0.7856 (13.7s)
[no shortcut (plain)] ep  14/20 loss 0.9331 val_acc 0.8534 (13.7s)
[no shortcut (plain)] ep  16/20 loss 0.8710 val_acc 0.8620 (13.7s)
[no shortcut (plain)] ep  18/20 loss 0.8331 val_acc 0.8872 (13.8s)
[no shortcut (plain)] ep  20/20 loss 0.8146 val_acc 0.8878 (13.7s)
Loaded saved run: task4_no_augmentation
[no augmentation] ep   1/20 loss 1.9002 val_acc 0.4328 (14.9s)
[no augmentation] ep   2/20 loss 1.4235 val_acc 0.6282 (14.9s)
[no augmentation] ep   4/20 loss 1.1175 val_acc 0.6844 (14.9s)
[no augmentation] ep   6/20 loss 0.9519 val_acc 0.7242 (14.9s)
[no augmentation] ep   8/20 loss 0.8567 val_acc 0.7400 (14.9s)
[no augmentation] ep  10/20 loss 0.7733 val_acc 0.7932 (14.9s)
[no augmentation] ep  12/20 loss 0.6854 val_acc 0.8180 (14.9s)
[no augmentation] ep  14/20 loss 0.5943 val_acc 0.8360 (14.9s)
[no augmentation] ep  16/20 loss 0.5414 val_acc 0.8620 (14.9s)
[no augmentation] ep  18/20 loss 0.5288 val_acc 0.8652 (14.9s)
[no augmentation] ep  20/20 loss 0.5262 val_acc 0.8654 (14.9s)
Loaded saved run: task4_no_cutout_flip_crop_only
[no cutout (flip+crop only)] ep   1/20 loss 1.9417 val_acc 0.4172 (15.0s)
[no cutout (flip+crop only)] ep   2/20 loss 1.5081 val_acc 0.4660 (15.0s)
[no cutout (flip+crop only)] ep   4/20 loss 1.1879 val_acc 0.6960 (15.0s)
[no cutout (flip+crop only)] ep   6/20 loss 1.0164 val_acc 0.7770 (15.0s)
[no cutout (flip+crop only)] ep   8/20 loss 0.9381 val_acc 0.7700 (15.0s)
[no cutout (flip+crop only)] ep  10/20 loss 0.8808 val_acc 0.8152 (15.0s)
[no cutout (flip+crop only)] ep  12/20 loss 0.8278 val_acc 0.8026 (15.0s)
[no cutout (flip+crop only)] ep  14/20 loss 0.7777 val_acc 0.8624 (14.9s)
[no cutout (flip+crop only)] ep  16/20 loss 0.7232 val_acc 0.8856 (14.9s)
[no cutout (flip+crop only)] ep  18/20 loss 0.6800 val_acc 0.9058 (15.0s)
[no cutout (flip+crop only)] ep  20/20 loss 0.6617 val_acc 0.9078 (15.0s)
Loaded saved run: task4_no_weight_decay
[no weight decay] ep   1/20 loss 1.9914 val_acc 0.4216 (15.0s)
[no weight decay] ep   2/20 loss 1.6413 val_acc 0.4904 (15.0s)
[no weight decay] ep   4/20 loss 1.3459 val_acc 0.6280 (15.0s)
[no weight decay] ep   6/20 loss 1.2145 val_acc 0.6412 (15.0s)
[no weight decay] ep   8/20 loss 1.1304 val_acc 0.7362 (15.0s)
[no weight decay] ep  10/20 loss 1.0617 val_acc 0.8034 (15.0s)
```

```
[no weight decay] ep  12/20 loss 1.0078 val_acc 0.8258 (15.0s)
[no weight decay] ep  14/20 loss 0.9635 val_acc 0.8430 (15.0s)
[no weight decay] ep  16/20 loss 0.9267 val_acc 0.8510 (15.0s)
[no weight decay] ep  18/20 loss 0.9089 val_acc 0.8602 (15.0s)
[no weight decay] ep  20/20 loss 0.9057 val_acc 0.8648 (15.0s)
Loaded saved run: task4_no_label_smoothing
[no label smoothing] ep   1/20 loss 1.8861 val_acc 0.4106 (15.0s)
[no label smoothing] ep   2/20 loss 1.4143 val_acc 0.5124 (15.0s)
[no label smoothing] ep   4/20 loss 1.0595 val_acc 0.5838 (15.0s)
[no label smoothing] ep   6/20 loss 0.9089 val_acc 0.6228 (15.0s)
[no label smoothing] ep   8/20 loss 0.7785 val_acc 0.7372 (14.9s)
[no label smoothing] ep  10/20 loss 0.6857 val_acc 0.8006 (14.9s)
[no label smoothing] ep  12/20 loss 0.6113 val_acc 0.7782 (15.0s)
[no label smoothing] ep  14/20 loss 0.5294 val_acc 0.8466 (15.0s)
[no label smoothing] ep  16/20 loss 0.4487 val_acc 0.8726 (14.9s)
[no label smoothing] ep  18/20 loss 0.3929 val_acc 0.8922 (15.0s)
[no label smoothing] ep  20/20 loss 0.3682 val_acc 0.8926 (15.0s)
Loaded saved run: task4_AdamW_instead_of_SGD
[AdamW instead of SGD] ep   1/20 loss 1.9790 val_acc 0.4682 (15.0s)
[AdamW instead of SGD] ep   2/20 loss 1.5646 val_acc 0.5468 (15.0s)
[AdamW instead of SGD] ep   4/20 loss 1.3106 val_acc 0.6788 (15.0s)
[AdamW instead of SGD] ep   6/20 loss 1.1752 val_acc 0.7126 (15.0s)
[AdamW instead of SGD] ep   8/20 loss 1.0827 val_acc 0.7760 (15.0s)
[AdamW instead of SGD] ep  10/20 loss 1.0207 val_acc 0.8168 (15.0s)
[AdamW instead of SGD] ep  12/20 loss 0.9722 val_acc 0.8348 (15.0s)
[AdamW instead of SGD] ep  14/20 loss 0.9286 val_acc 0.8488 (15.0s)
[AdamW instead of SGD] ep  16/20 loss 0.8940 val_acc 0.8664 (15.0s)
[AdamW instead of SGD] ep  18/20 loss 0.8803 val_acc 0.8702 (15.0s)
[AdamW instead of SGD] ep  20/20 loss 0.8752 val_acc 0.8710 (15.0s)
```

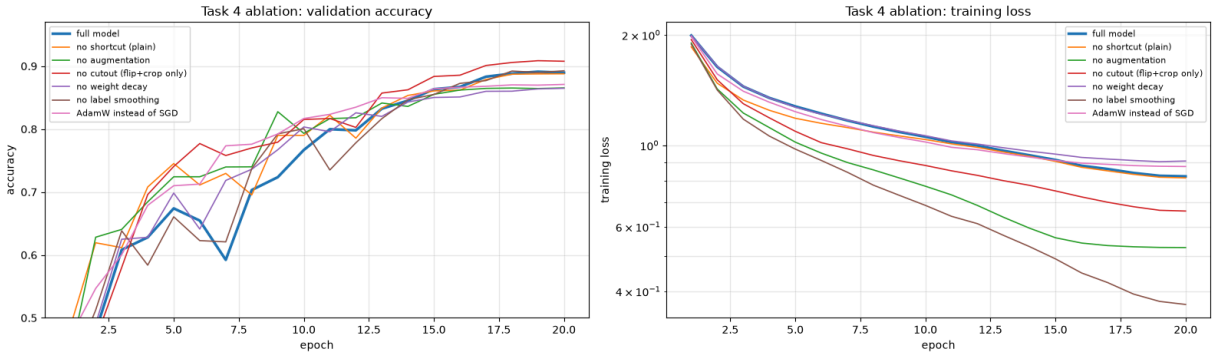
In [19]:

```
base = R4["full model"]["val_acc"][-1]
t4 = pd.DataFrame({k: {"parameters": H["params"], "val acc": H["val_acc"][-1],
                       "change vs full model": H["val_acc"][-1] - base, "best val
acc": max(H["val_acc"]),
                       "final train loss": H["train_loss"][-1]} for k, H in
R4.items()}).T
print("Ablation, final epoch (20), validation split:")
display(t4.style.format({"parameters": "{:,.0f}", "val acc": "{:.2%}", "change vs
full model": "{:+.2%}",
                         "best val acc": "{:.2%}", "final train loss": "{:.4f}"})
       .highlight_max(subset=["val acc"], color="#c6efce"))

fig, ax = plt.subplots(1, 2, figsize=(16, 4.8))
for i, (k, H) in enumerate(R4.items()):
    ep = np.arange(1, len(H["val_acc"]) + 1)
    lw = 2.5 if k == "full model" else 1.2
    ax[0].plot(ep, H["val_acc"], color=cmap(i), lw=lw, label=k)
    ax[1].plot(ep, H["train_loss"], color=cmap(i), lw=lw, label=k)
ax[0].set(xlabel="epoch", ylabel="accuracy", title="Task 4 ablation: validation
accuracy", ylim=(0.5, 0.97))
ax[1].set(xlabel="epoch", ylabel="training loss", title="Task 4 ablation: training
loss", yscale="log")
for a in ax:
    a.grid(alpha=0.3, which="both"); a.legend(fontsize=8)
plt.tight_layout(); savefig("task4_ablation"); plt.show()
```

Ablation, final epoch (20), validation split:

| | parameters | val acc | change vs full model | best val acc | final train loss |
|---|---|---|---|---|---|
| **full model** | 460,548 | 88.96% | +0.00% | 89.12% | 0.8228 |
| **no shortcut (plain)** | 453,476 | 88.78% | -0.18% | 88.78% | 0.8146 |
| **no augmentation** | 460,548 | 86.54% | -2.42% | 86.54% | 0.5262 |
| **no cutout (flip+crop only)** | 460,548 | 90.78% | +1.82% | 90.86% | 0.6617 |
| **no weight decay** | 460,548 | 86.48% | -2.48% | 86.48% | 0.9057 |
| **no label smoothing** | 460,548 | 89.26% | +0.30% | 89.26% | 0.3682 |
| **AdamW instead of SGD** | 460,548 | 87.10% | -1.86% | 87.10% | 0.8752 |




**Interpretation:** The full model reaches **88.96%** validation accuracy.

- **Cutout:** removing cutout gives the best result, **90.78%** (+1.8 points). With cutout the training loss stays higher (0.82 against 0.66). So the model underfits in 20 epochs. Cutout seems to need a longer training schedule.
- **Augmentation:** without any augmentation the accuracy drops to 86.54% (-2.4). The training loss is the lowest of all runs with label smoothing (0.53). So the model memorises the training set.
- **Weight decay:** without weight decay the accuracy drops to 86.48% (-2.5), and the training loss goes *up* (0.91). With batch norm, weight decay keeps the weights small. This keeps the effective learning rate high. Without it, training slows down.
- **Optimizer:** AdamW (learning rate 0.002, weight decay 0.05) reaches 87.10% (-1.9). This is lower than SGD with Nesterov momentum.
- **Shortcut:** removing the shortcuts costs only 0.2 points at 14 layers. This agrees with Task 3(b), where shallow networks show only a small difference.
- **Label smoothing:** removing it changes the accuracy by +0.3 points. This is within the normal variation between runs.

Label smoothing 0.1 keeps the training loss above about 0.5. So I compare training losses only between runs that use it. Following the selection rule, the final model is the residual network **without cutout** (flip + crop, weight decay, label smoothing, SGD with Nesterov momentum). It reaches 90.78% validation accuracy.

## Final test evaluation

I load the saved weights of the selected model and evaluate them once on the 10,000 test images. The notebook saves the result, so it does not evaluate the test set again on a re-run.

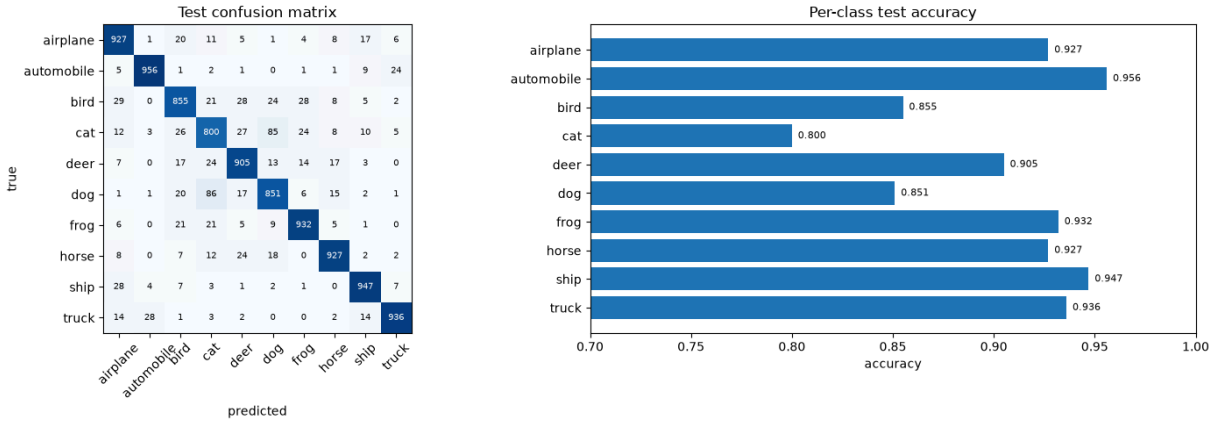
In [20]:
```
FINAL = max(R4, key=lambda k: R4[k]["val_acc"][-1])
print("selected on validation:", FINAL, f"(val acc {R4[FINAL]['val_acc']
[-1]:.4f})")


def test_once():
    cfg = VARIANTS_4[FINAL]
    model = CifarNet(**{**T4_ARCH, "residual": cfg.get("residual",
True)}).to(device)
    model.load_state_dict(torch.load(f"{CKPT}/task4_{FINAL.replace(' ', '_')}.pt",
map_location=device))
    model.eval()
    preds = []
    with torch.no_grad():
        for i in range(0, len(y_te), 1000):
            with torch.autocast(device.type, dtype=torch.float16, enabled=AMP):
                preds.append(model(X_te[i:i + 1000].to(device)).argmax(1).cpu())
    preds = torch.cat(preds)
    cm = torch.zeros(10, 10, dtype=torch.long)
    for t, p in zip(y_te, preds):
        cm[t, p] += 1
    return dict(model=FINAL, test_acc=(preds == y_te).float().mean().item(),
confusion=cm.tolist())


T4_TEST = cached("task4_test", test_once)
print(f"Final model: {T4_TEST['model']}  |  parameters {T4_PARAMS:,}  |  TEST
ACCURACY = {T4_TEST['test_acc']:.4f}")

cm = np.array(T4_TEST["confusion"])
fig, ax = plt.subplots(1, 2, figsize=(15, 5))
im = ax[0].imshow(cm / cm.sum(1, keepdims=True), cmap="Blues", vmin=0, vmax=1)
for i in range(10):
    for j in range(10):
        ax[0].text(j, i, cm[i, j], ha="center", va="center", fontsize=7,
color="white" if i == j else "black")
ax[0].set_xticks(range(10)); ax[0].set_xticklabels(CLASSES, rotation=45);
ax[0].set_yticks(range(10))
ax[0].set_yticklabels(CLASSES); ax[0].set(xlabel="predicted", ylabel="true",
title="Test confusion matrix")
per_class = cm.diagonal() / cm.sum(1)
ax[1].barh(CLASSES, per_class, color="tab:blue"); ax[1].set_xlim(0.7, 1.0)
for i, v in enumerate(per_class):
    ax[1].text(v + 0.003, i, f"{v:.3f}", va="center", fontsize=8)
ax[1].set(title="Per-class test accuracy", xlabel="accuracy");
ax[1].invert_yaxis()
plt.tight_layout(); savefig("task4_test"); plt.show()
```

```
selected on validation: no cutout (flip+crop only) (val acc 0.9078)
Loaded saved run: task4_test
Final model: no cutout (flip+crop only)  |  parameters 460,548  |  TEST ACCURACY =
0.9036
```




**Interpretation:** The final model reaches **90.36% test accuracy** with 460,548 parameters and 20 training epochs. This is 0.4 points below its validation accuracy (90.78%). So choosing among seven variants on the validation split did not overfit much.

The vehicle classes are the easiest: automobile 95.6%, ship 94.7% and truck 93.6%. The hardest classes are **cat (80.0%) and dog (85.1%)**. The model mostly confuses these two with each other: it predicts 85 cats as dog and 86 dogs as cat. It also confuses bird (85.5%) with airplane, deer and frog.